

DASC7606

Deep Learning

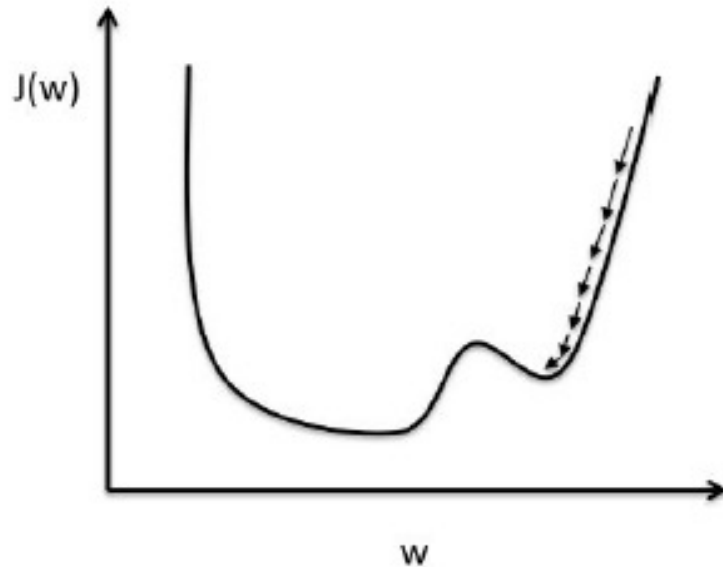
3: Convolutional Neural Networks (CNNs) and Computer Vision

Dr Bethany Chan
Professor Francis Chin
2025/26

NN Training – Outline

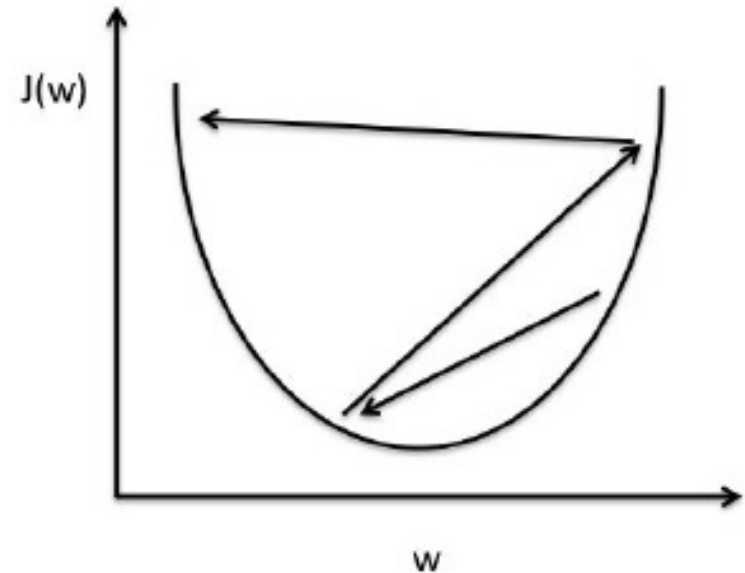
1. Gradient Descent, SGD and Mini-batch SGD
2. Vanishing Gradient Problem
3. Not Zero-Centered Problem
4. Choice of Activation Function
5. Learning Rate Problems
6. Momentum and Nesterov Momentum
7. Adaptive Learning Rates
8. Overfitting Problem
9. Weight Regularization, Dropout, Early Stopping
10. Network Distillation
11. Batch Normalization

5. Learning Rate Problem



Small learning rate:

- Many iterations till convergence
- Trapped in local minimum

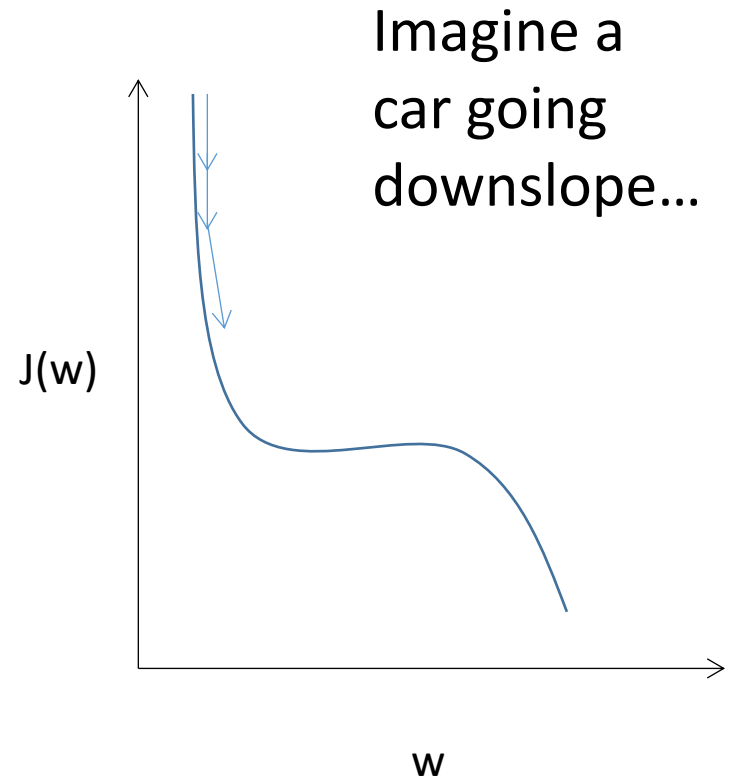


Large learning rate:

- Overshooting
- No convergence

6. Momentum

- Trapped in a local minimum:
What if you get stuck in a flattish region where gradient is small?
- Let past steps affect current step so that you **keep on moving in the direction you were heading**
- Weights updated by a function of past steps and current gradient



Momentum

- Weights updated by a function of past steps and current gradient
- Introduce new variable to capture past steps:

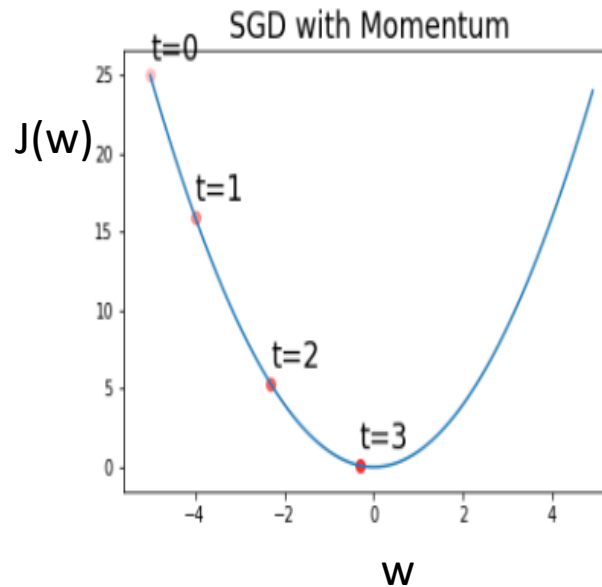
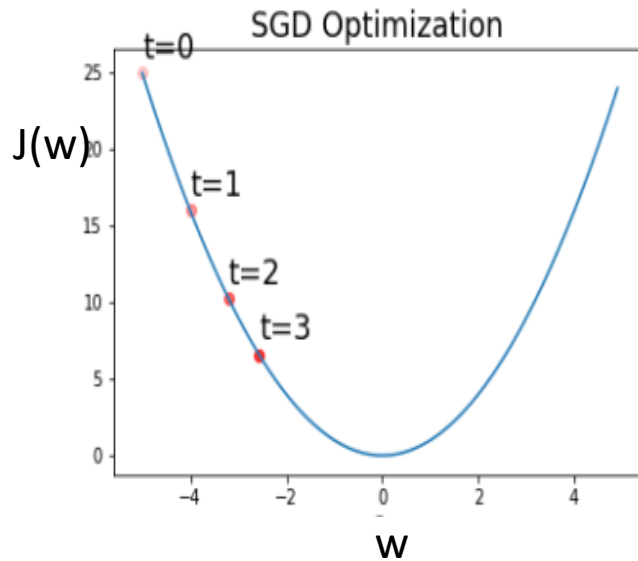
v_t = velocity at time t

- Update of weights: $w_t \leftarrow w_{t-1} + v_t$
- Update of velocity:

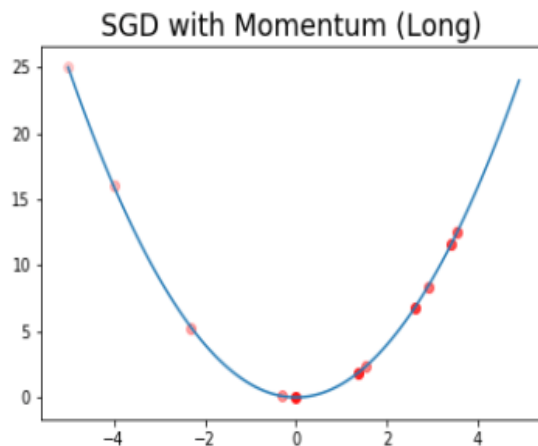
$$v_t \leftarrow \mu v_{t-1} - \alpha \nabla J(w_{t-1})$$

$$[\text{with } v_1 = -\alpha \nabla J(w_0)]$$

- Momentum step: μv_{t-1} [μ is typically set to 0.9]
- Gradient step: $-\alpha \nabla J(w_{t-1})$



Momentum
gets to minimum
faster



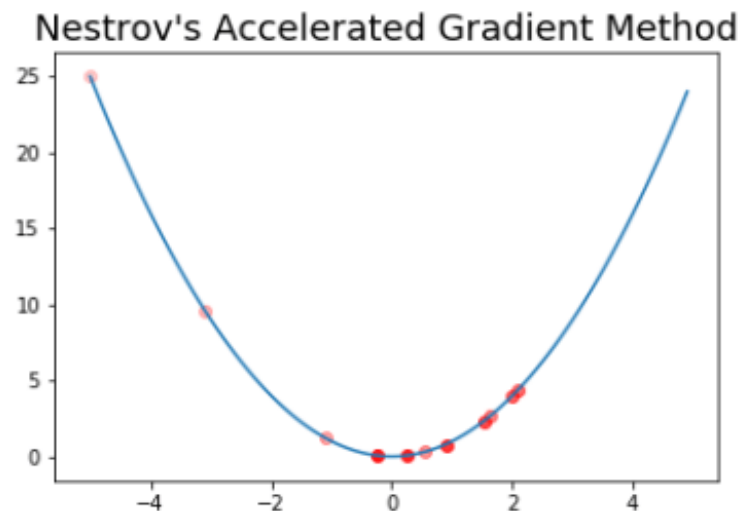
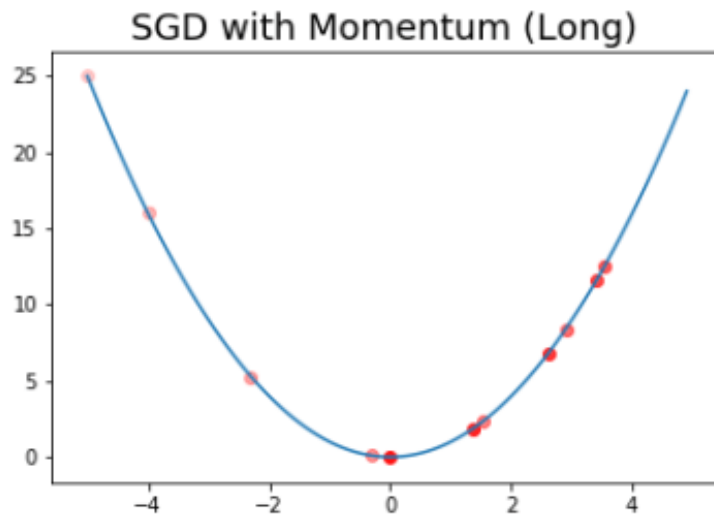
But overshoots
the minimum
if run too long

Wouldn't it be
great if we could:
detect the
overshooting, i.e.
change in the
direction of the
gradient earlier?

Nesterov Momentum

- $v_t \leftarrow \mu v_{t-1} - \alpha \nabla J(w_{t-1})$
- $w_t \leftarrow w_{t-1} + v_t$

- $v_t \leftarrow \mu v_{t-1} - \alpha \nabla J(w_{t-1} + \mu v_{t-1})$
- $w_t \leftarrow w_{t-1} + v_t$

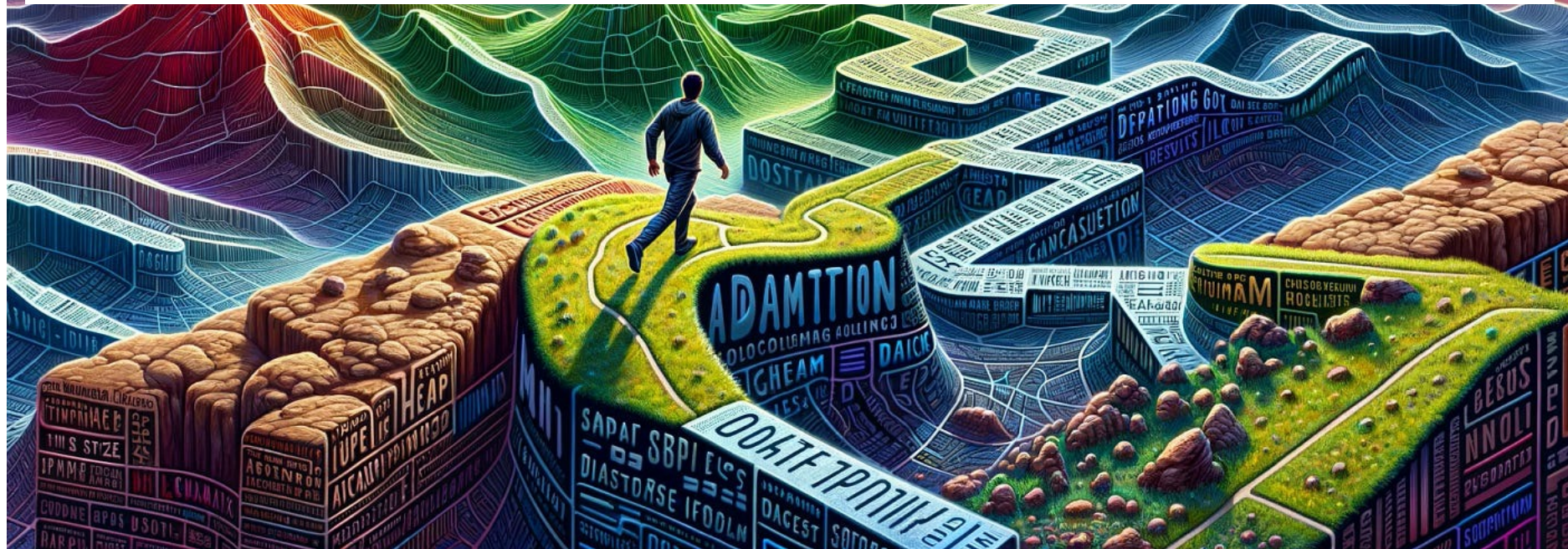


7. Adaptive Learning Rates

Navigating a Complex Terrain

<https://towardsdatascience.com/the-math-behind-adam-optimizer-c41407efe59b>

Large strides for some area and cautious steps for others
Take small steps when gradient changes rapidly and unpredictably
large when gradient changes slowly or predictably
Maintain individual learning rates for each parameter in your model
Rates adjusted as training progress, based on the data encountered



Adaptive Learning Rates: AdaGrad

- **Adaptive Gradient**

Historical gradients for w	Learning rate for w
High	Low
Low	High

- Idea:

- Introduce variable **G** to capture historical gradient information (second moment), with the following update rule :

$$\mathbf{G}_t \leftarrow \mathbf{G}_{t-1} + (\nabla J(\mathbf{w}_{t-1}))^2$$

- Update weights:

$$\mathbf{w}_t \leftarrow \mathbf{w}_{t-1} - \frac{\alpha}{\sqrt{\mathbf{G}_{t-1} + \epsilon}} \nabla J(\mathbf{w}_{t-1})$$

ϵ (epsilon) is a small scalar (e.g., 10^{-8}) added to prevent division by zero and maintain numerical stability.

- Learning rate diminishes quickly as G gets bigger and bigger

Adaptive Learning Rates: RMSProp

AdaGrad

$$G \leftarrow G + (\nabla J(w))^2$$

G gets bigger and bigger

$$w \leftarrow w - \frac{\alpha}{\sqrt{G+\epsilon}} \nabla J(w)$$

coefficient gets smaller and smaller

RMSProp

$$G \leftarrow \beta G + (1-\beta) (\nabla J(w))^2$$

β is the decay rate (typically 0.9, 0.99,..) preventing G from getting too big

$$w \leftarrow w - \frac{\alpha}{\sqrt{G+\epsilon}} \nabla J(w)$$

Same update rule for w but problem goes away

Adaptive Learning Rates: Adam

RMSProp

$$G \leftarrow \beta G + (1-\beta) (\nabla J(w))^2$$

G does not get too big

$$w \leftarrow w - \frac{\alpha}{\sqrt{G+\epsilon}} \nabla J(w)$$

Same update rule as AdaGrad

Adam

$$v \leftarrow \beta v + (1-\beta) (\nabla J(w))^2$$

Same idea as RMSProp (except called v instead of G)

$$m \leftarrow \gamma m + (1-\gamma) (\nabla J(w))$$

New first order variable m introduced to “smooth” gradient

$$w \leftarrow w - \frac{\alpha}{\sqrt{v+\epsilon}} m$$

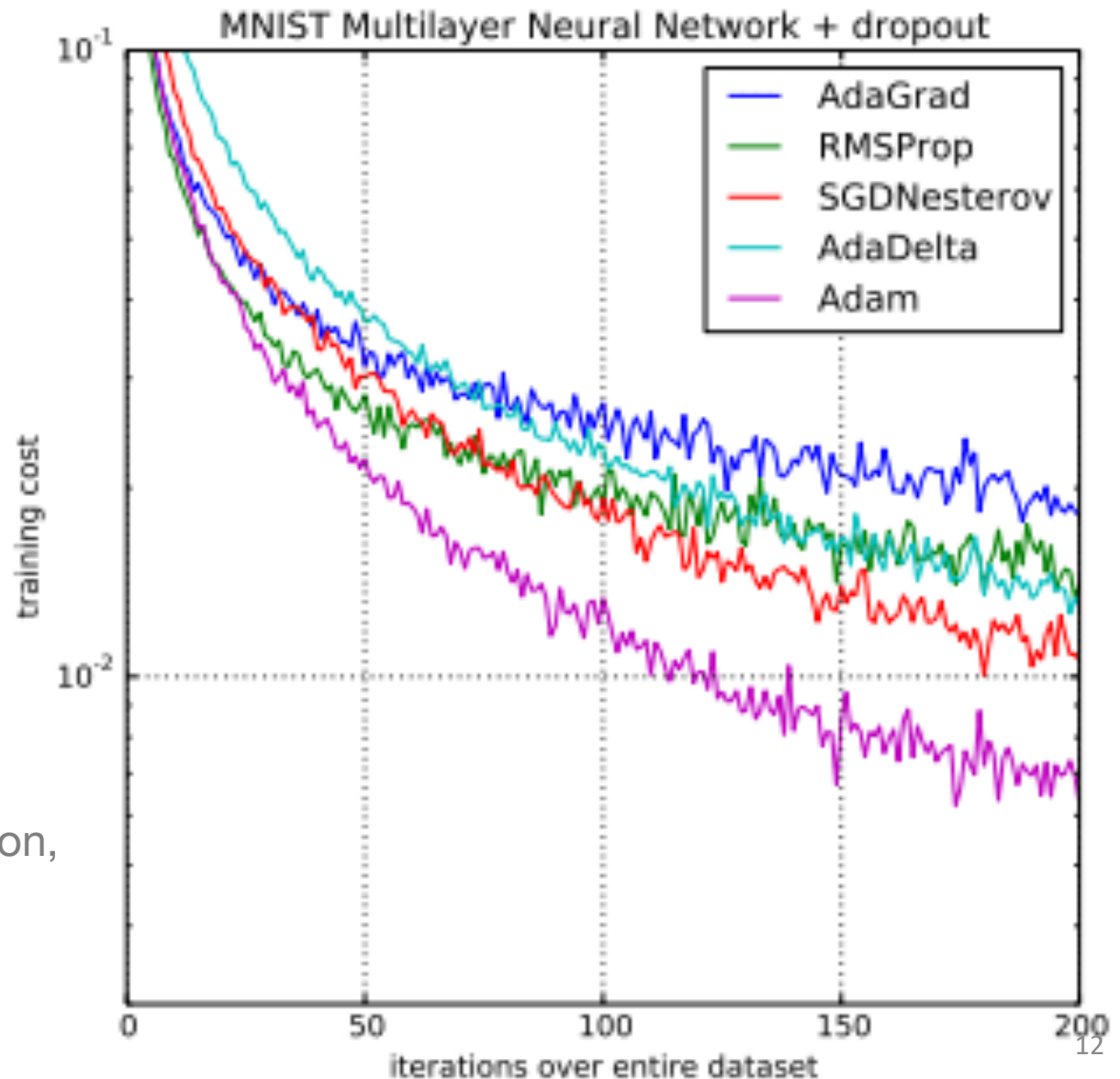
Both v and m used to update w

Parameter values: $\beta=0.999$, $\gamma=0.9$, $\alpha=0.001$ (10^{-5} , 0.3), $\epsilon=10^{-8}$

Adaptive Learning Rates: Performance

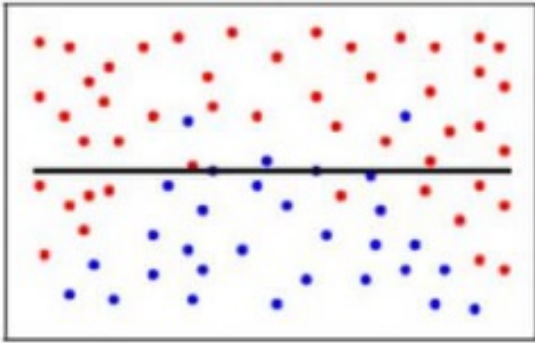
Comparison of
Adam to Other
Optimization
Algorithms
Training a
Multilayer
Perceptron

Taken from
Adam: A Method for
Stochastic Optimization,
2015.

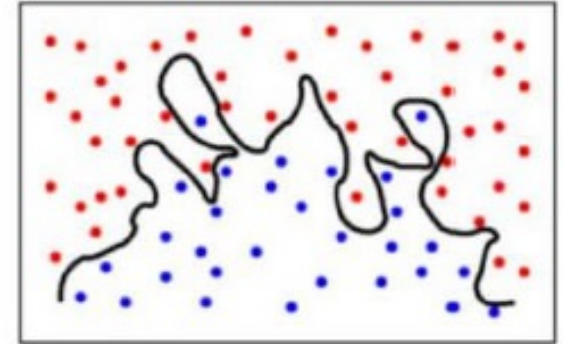


8. Overfitting Problem

Underfitting



Overfitting



NN with a few neurons

NN with many layers and neurons

Difficult to find the right model

Approach:

Start with an overly complicated model

Then apply certain techniques to prevent overfitting

(Analogy: buying clothing without knowing the size)

Overfitting Problem: familiar concept



Memorization vs Generalization

9. Weight Regularization

- Observation: Large weights $\rightarrow$ overfitting
- Add the size of the weights to cost function

$$\min \frac{1}{M} \sum_i J(h_{\theta}(\mathbf{x}_i), y_i) + \lambda \sum_j |\theta_j|$$

L1 regularization
(Lasso Regression)

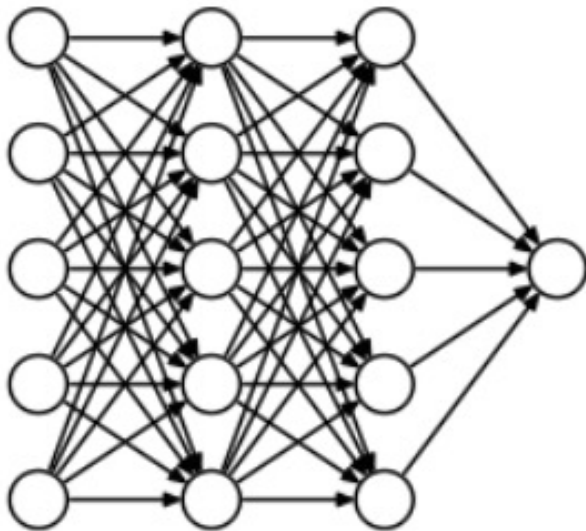
Sparse output (lots of zeros) $\rightarrow$ feature selection

$$\min \frac{1}{M} \sum_i J(h_{\theta}(\mathbf{x}_i), y_i) + \lambda \sum_j \theta_j^2$$

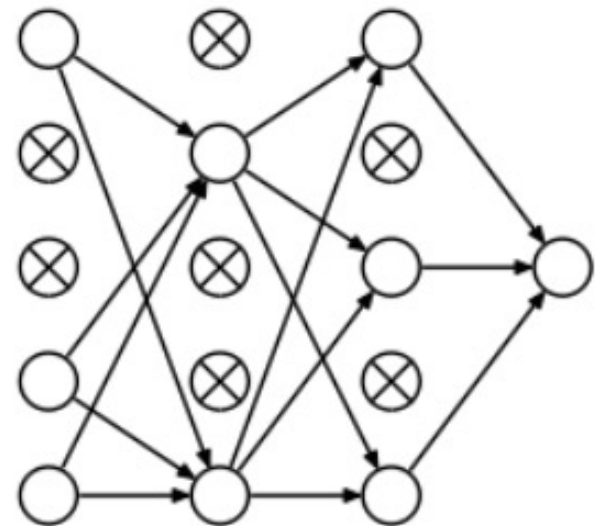
L2 regularization
(Ridge Regression)

Computationally efficient (analytical solutions)

Dropout: random



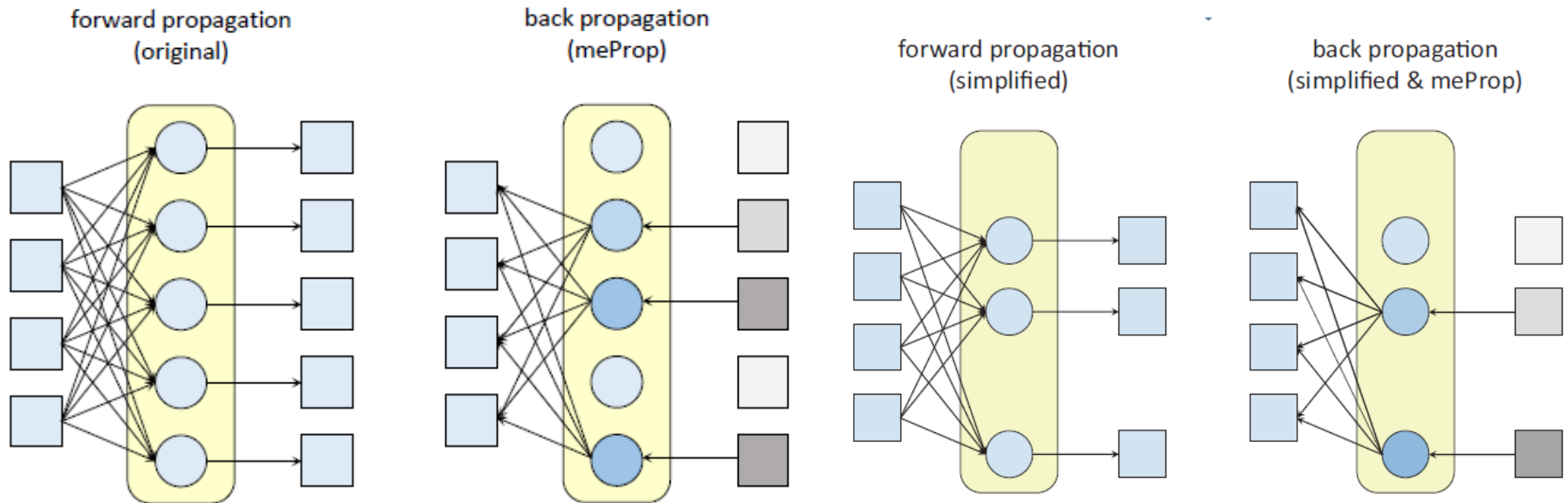
Standard Neural Network



**After applying dropout –
Randomly select 50% to
drop out, prevents reliance
on any one neuron**

Dropout: not random

Ref: Minimal Effort Back Propagation Method
Work of Xu SUN et. al. of Peking University (Nov 2017)

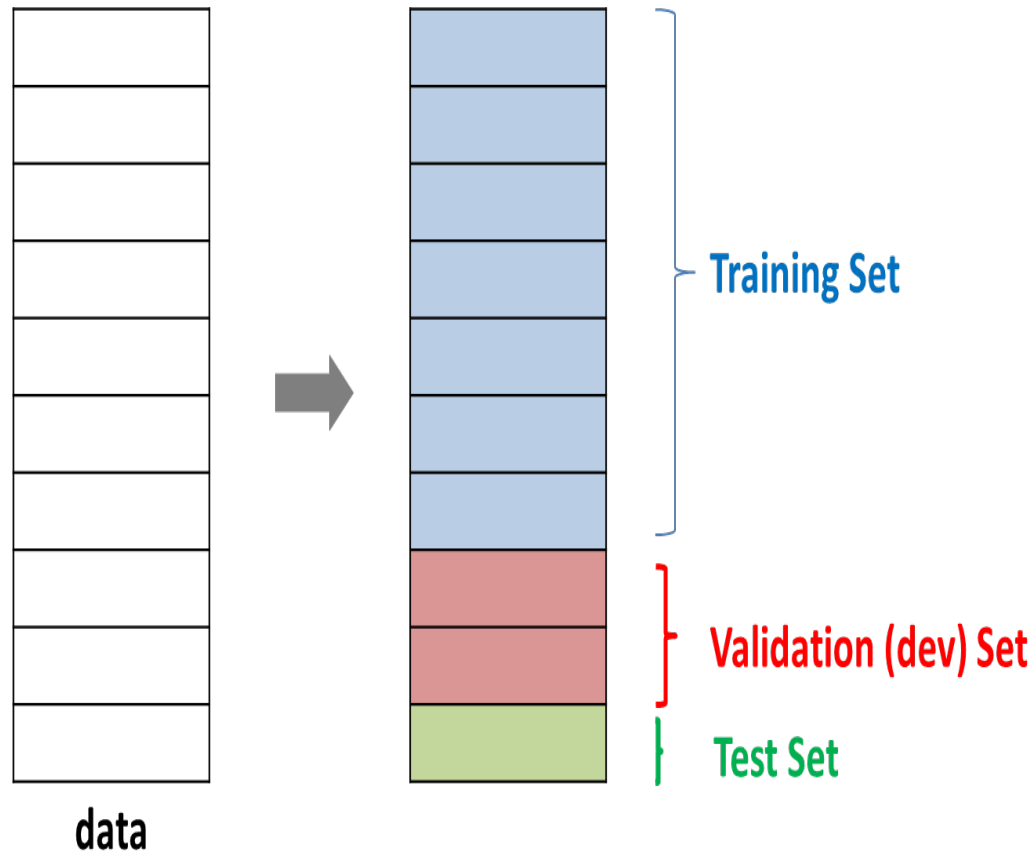


Model is trained for several iterations. Keep track of the “activeness” of path – indicated by shade of neurons.

Model simplification – inactive neurons are eliminated, repeat as needed

Early Stopping

Don't give the network time to overfit



STOP!!! when accuracy stops improving

Early Stopping

Don't give the network time to overfit



data

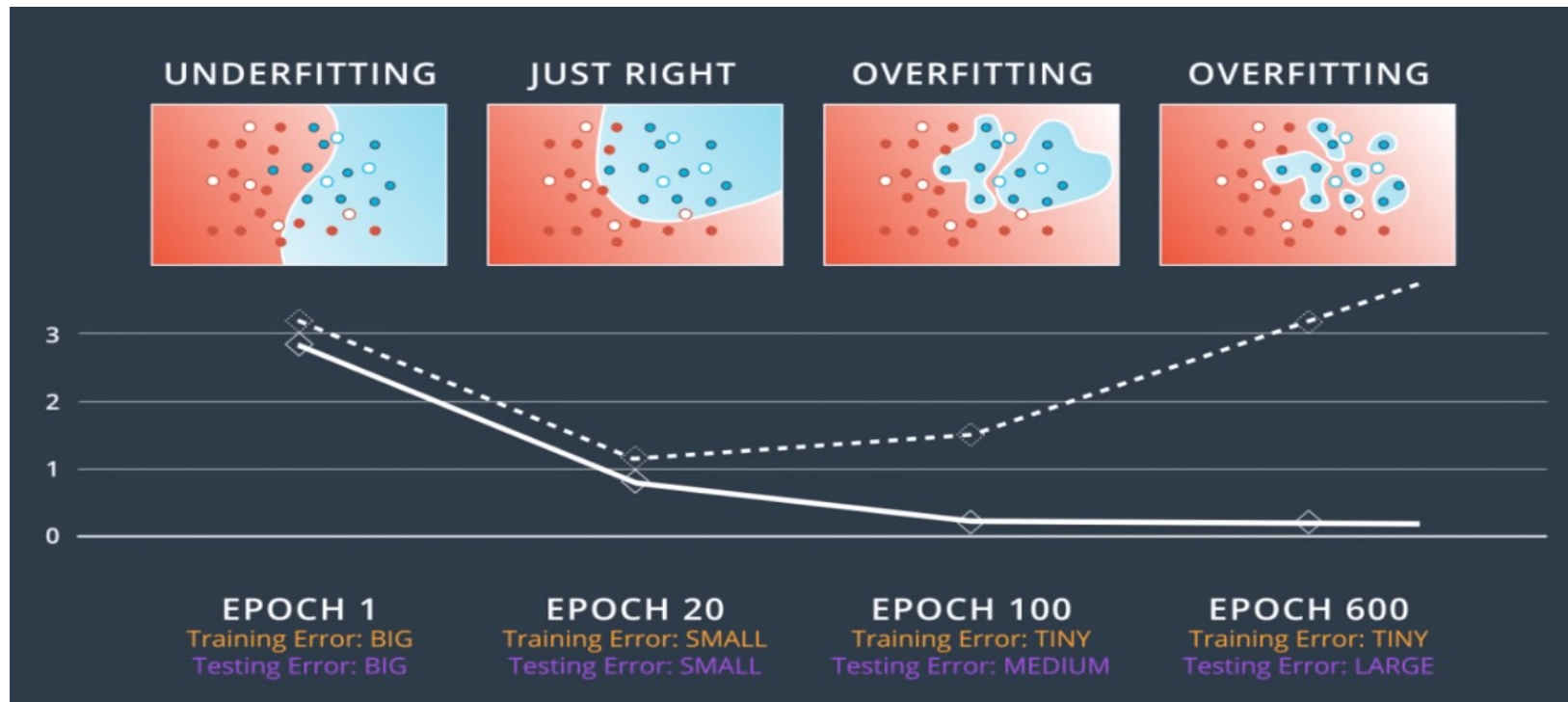


- Epoch 15: Train: 85% Validation: 80%
- Epoch 16: Train: 87% Validation: 82%
- Epoch 17: Train: 90% Validation: 85%
- Epoch 18: Train: 95% Validation: 83%
- Epoch 19: Train: 97% Validation: 78%
- Epoch 20: Train: 98% Validation: 75%

STOP!!! when accuracy stops improving

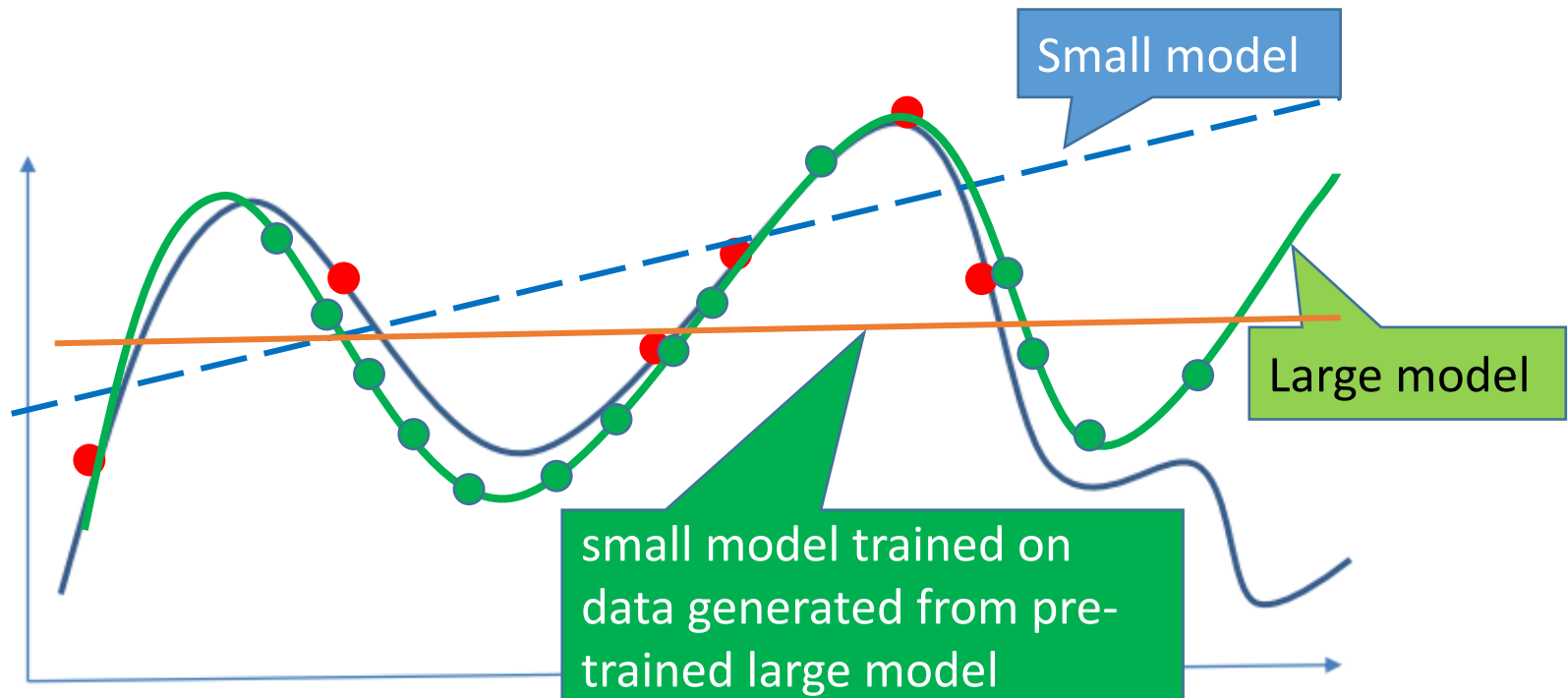
Early Stopping: training phenomenon

- Start with a complicated network architecture (model)
- Evaluate model by plotting the error of training dataset and validation dataset for each epoch
- Validation set error decreases as model generalizes until it gets to a ***minimum point — the Goldilocks spot*** (stops under-fitting and starts over-fitting)



10. Network Distillation

Train a small model (student) to **produce the same output** as the large model (tutor, generator of lots of noisy data)



11. Batch Normalization: preliminaries

Normalization

- features don't have the same range of values (number of bedrooms in flat vs square footage of flat)
- convert inputs to be on same range of values, e.g. [0,1]
- normalization:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$



Standardization

- make the input have mean 0 and standard deviation 1
- standardization:

$$x' = \frac{x - \text{mean}(x)}{\text{std}(x)}$$



Batch Normalization

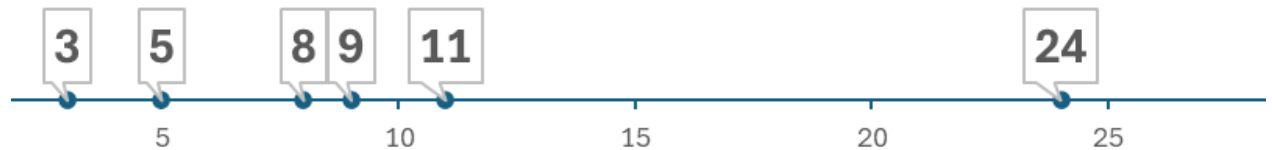
2 steps:

1. After calculating minibatch mean and std, standardize
2. Scale by γ (to be learned) and offset by β (to be learned)

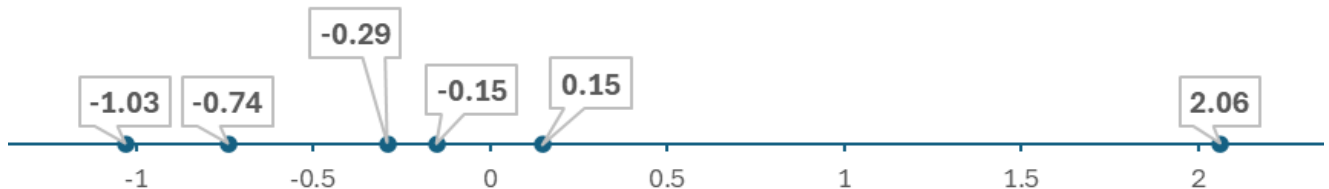
Original data:

Mean = 10

Std ≈ 6.78



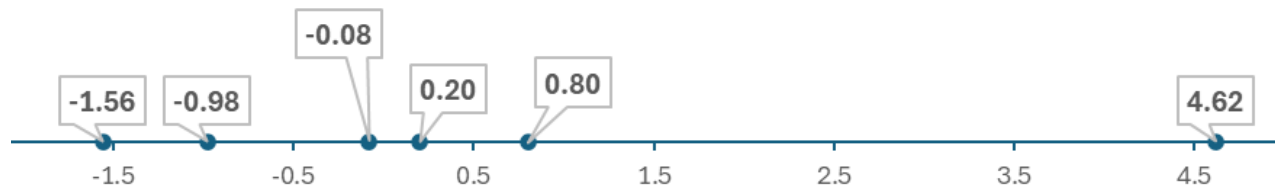
After standardize:



After scale by 2:

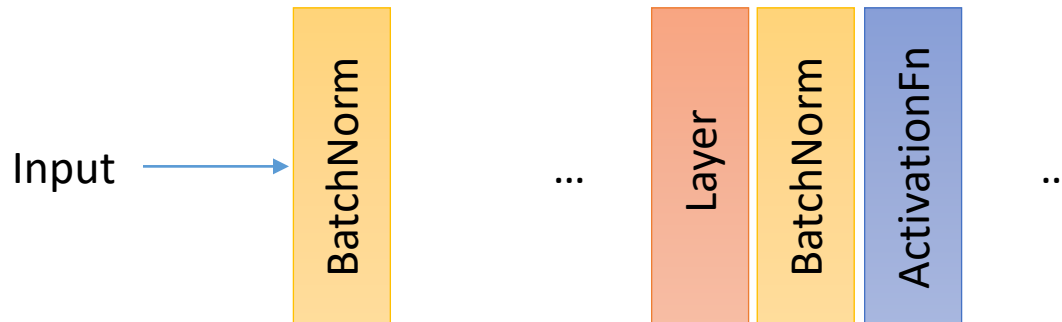


After offset by 0.5:

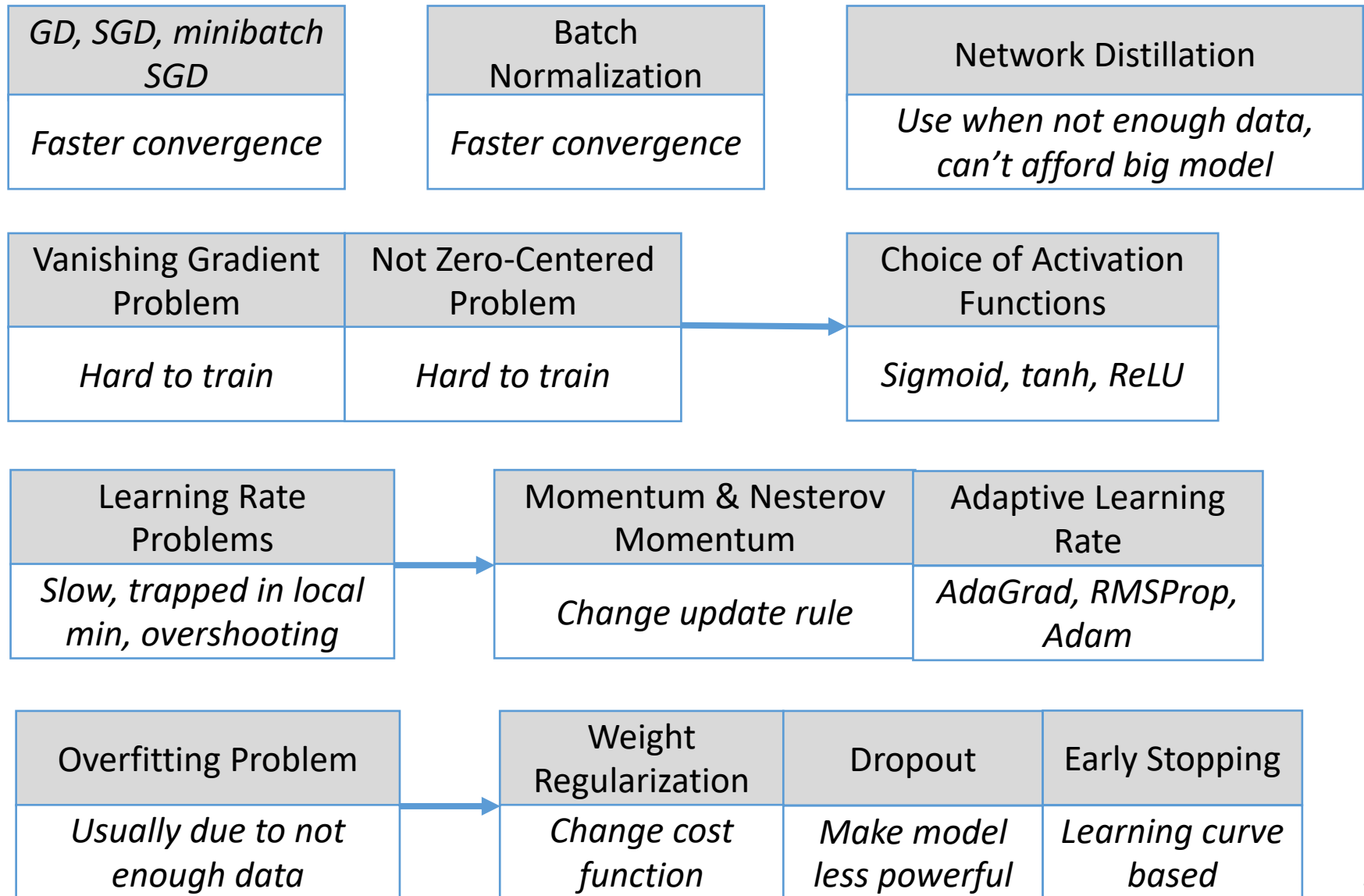


Batch Normalization

- Longer time to do one epoch
- But faster to achieve accuracy
- No need to normalize/standardize input – just use a BatchNorm
- Due to offset, bias in convolution layer may not be needed



NN Training – Mind Map



Convolutional Neural Networks (CNNs) and Computer Vision

Convolutional Neural Networks (CNNs)

1. What is Computer Vision?

2. Milestones in Computer Vision

3. What is Convolutional Neural Networks (CNNs)?

a. Convolutional layer – Filter

b. Pooling

c. Dense layer

4. Image Classification

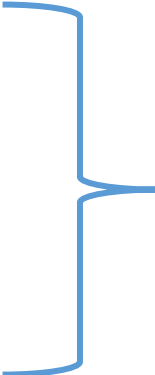
5. AlexNet 2012 and ZFNet 2013

6. VGNet 2014

7. GoogLeNet 2014

8. ResNet 2015

9. ReduNet 2022



ImageNet

References

1. MIT online course 6.S191 “Introduction to Deep Learning”
2. Stanford CS230 “Deep Learning”
3. Chan, Yu, You, Qi, Wright, Ma, ReduNet: A White-box Deep Network from the Principle of Maximizing Rate Reduction, Journal of Machine Learning Research 23 (2022) 1-103

1. Computer Vision

Applications:

- Image recognition and video search
- Security – Face and iris recognition, finger prints, intruder detection, ...
- Medical image processing
- Robotics
- Self-driving cars

Great increase of visual data (smartphone, video...)

50 Video Statistics You Can't Ignore In 2025

<https://www.synthesia.io/post/video-statistics>

- No. of internet users increases by 10% every year.
- Video made up 82% of all internet traffic in 2022.

Human vision seems easy, why?

Data through Evolution

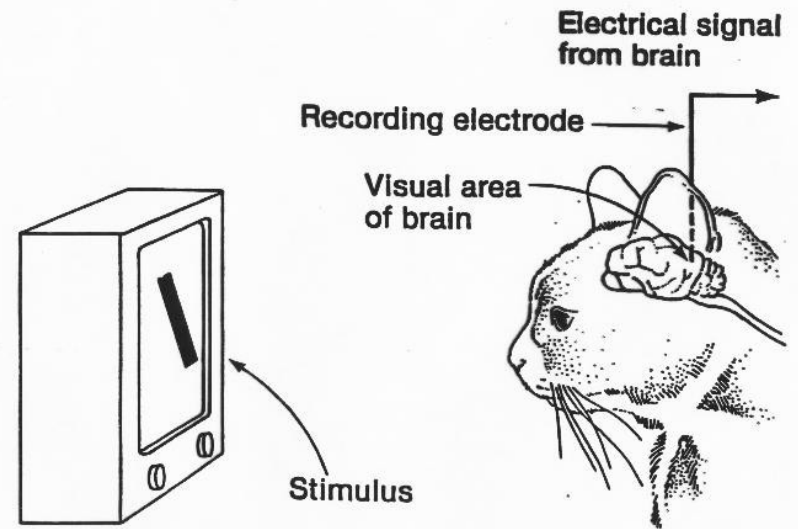
(ref: Han Moravec, Mind Children 1988)

- Visual perception: 540 million years of data
- Bipedal movement: 230+ million years
- Abstract thought: 100 thousand years

Human vision – *structure is instructive and inspiring*
8 million cortical neurons + 2 billion synapses

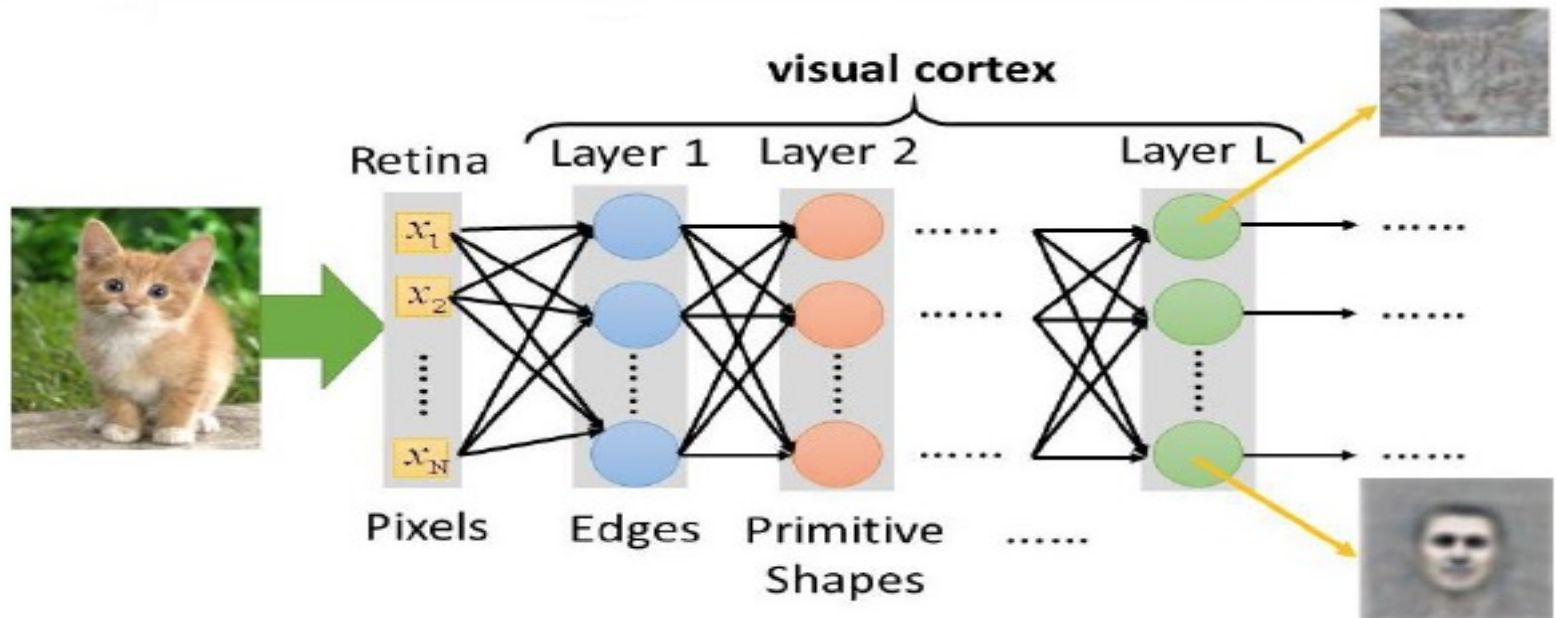
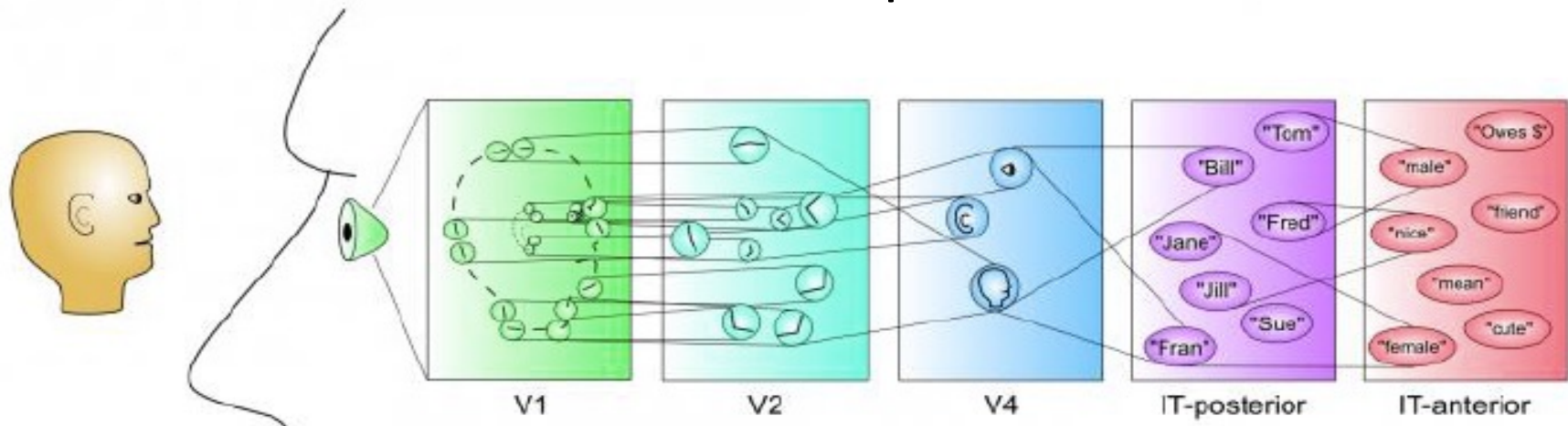
Hubel & Wiesel (H&W) 1959:
study on how neurons in
the visual cortex of a cat
respond to visual stimuli.

**Nobel prizewinning (1981)
key to understanding vision**



Visual Cortex –

vision starts with the response of simple cells to basic stimuli which is processed further.



Computer Vision is Hard

Viewpoint variation



Scale variation



Deformation



Occlusion



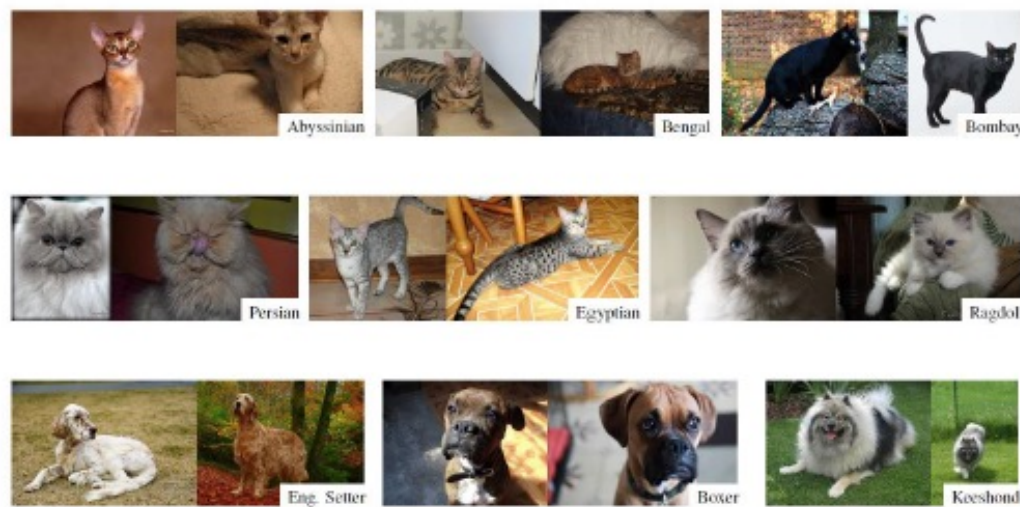
Illumination conditions



Background clutter



Intra-class variation



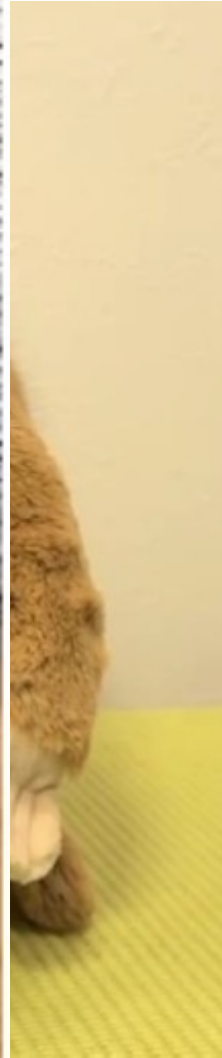
Occlusion



Disguise



Special Feature Proper Scaling



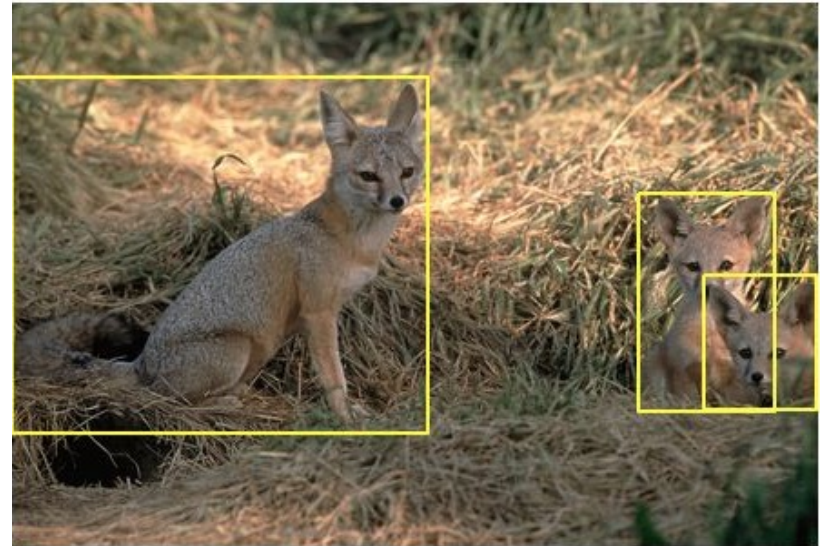
Other Challenges



How many bananas?

F.Wu, et.al., Detection and counting of banana bunches by integrating deep learning and classic image-processing algorithms, 2023

Object counting in video



Where do foxes end and does the grass begin?



Semantic segmentation.



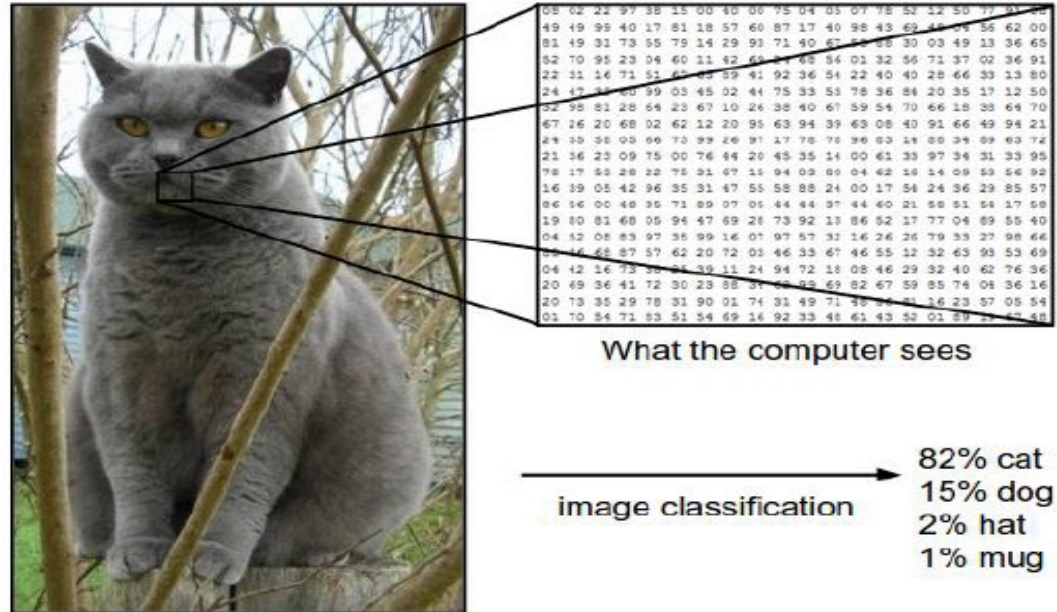
sky tree road grass water bldg mngn fg obj

Convolutional Neural Networks (CNNs)

1. What is Computer Vision?
- 2. Milestones in Computer Vision**
3. What is Convolutional Neural Networks (CNNs)?
 - a. Convolutional layer – Filter
 - b. Pooling
 - c. Dense layer
4. AlexNet 2012 and ZFNet 2013
5. VGNet 2014
6. GoogLeNet 2014
7. ResNet 2015
8. ReduNet 2022
9. References
 1. MIT online course 6.S191 “Introduction to Deep Learning”
 2. Stanford CS230 “Deep Learning”
 3. Chan, Yu, You, Qi, Wright, Ma, ReduNet: A White-box Deep Network from the Principle of Maximizing Rate Reduction, Journal of Machine Learning Research 23 (2022) 1-103

2. Milestones in Computer Vision

1959 – Russell Kirsch,
inventor of pixel,
developer of the first
digital image scanner



Lawrence Roberts
1963 Ph.D. thesis
on **deriving 3D
information** from
2D photographs.

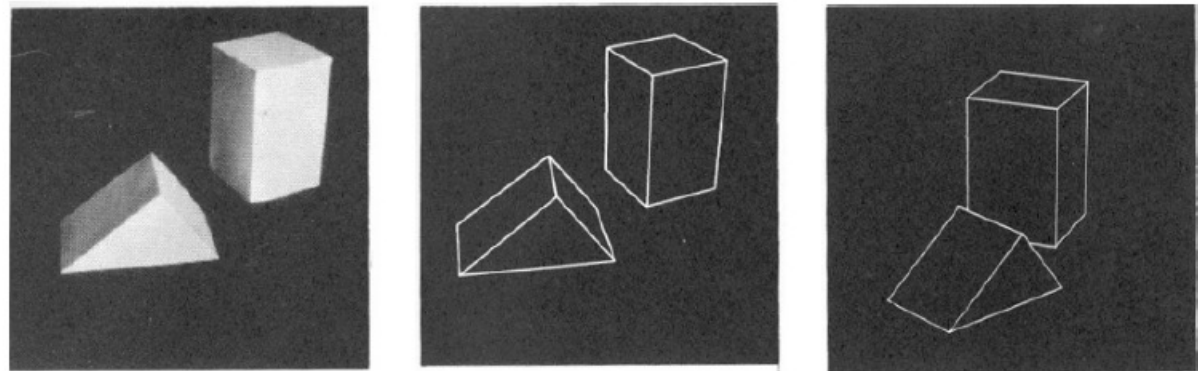


Figure: Original pic. (left), Line Drawing (middle), Rotated view (right).

- 1980, Kunihiro Fukushima (inspired by H&W) recognizing patterns unaffected by position and shifts (*convolutional layers*)
- 1982, David Marr (based on H&W) *hierarchical vision processing*, i.e., edges, curves, corners, ...
- 1997, J Shi and K. Malik (*image segmentation*) group related pixels together
- 1998, Yann LeCun (LeNet, MNIST handwritten digits) *backprop style learning algorithm* on convolutional NN architecture for *character recognition*
- Around 1999, stopped reconstructing 3D models for objects (proposed by Marr) and instead used *feature-based object recognition*, i.e., features invariant to image scaling, translation and rotation.



Problem: recognizing handwritten digits

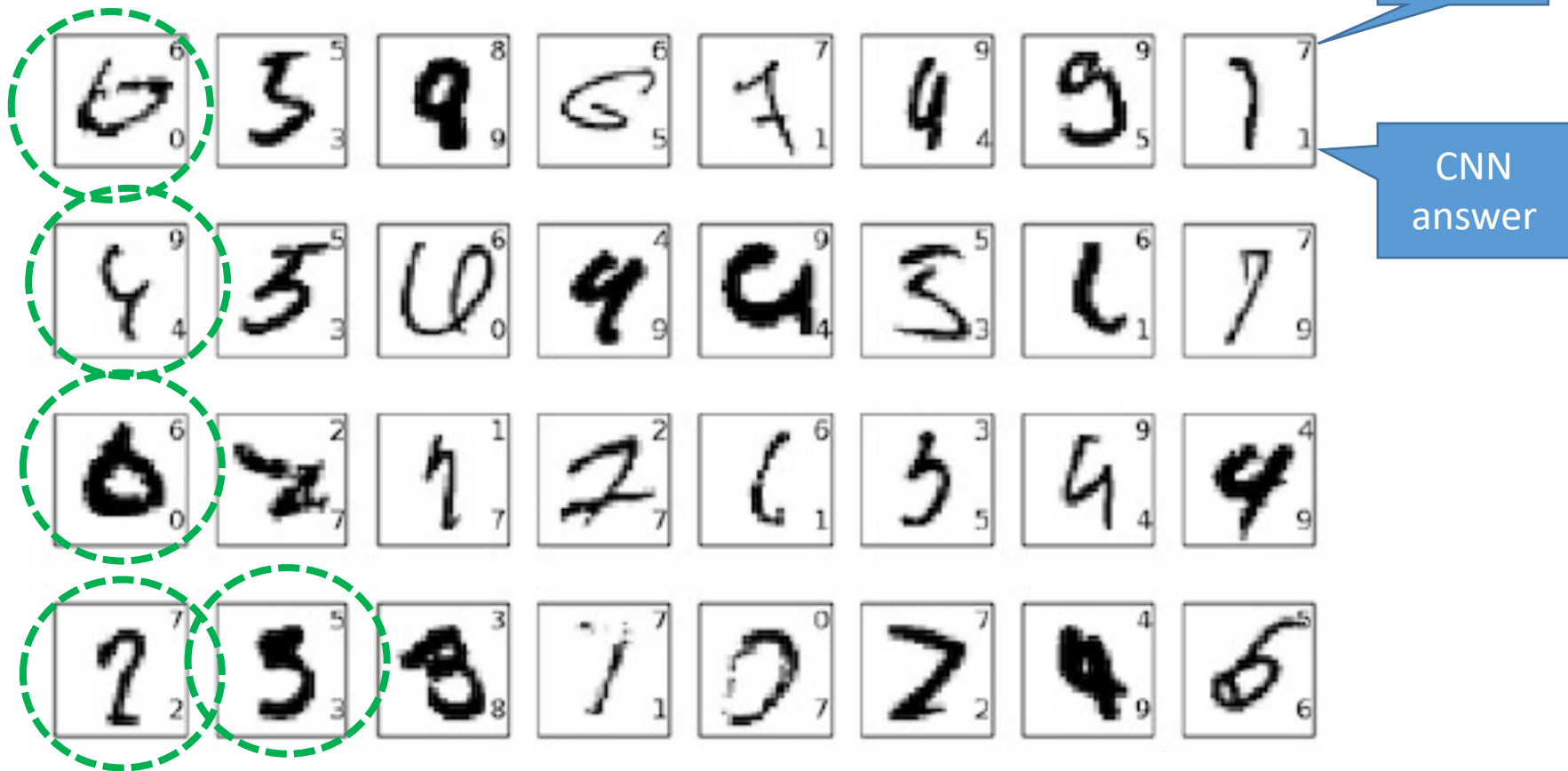
MNIST Dataset (1998):

- 60,000 images scanned from 250 people used as training data
- 10,000 images used as test data
- 28x28 pixel images
- each pixel takes a value in $[0; 255]$ representing the intensity (grey level)



CNN works better than human

Some are difficult even for humans:

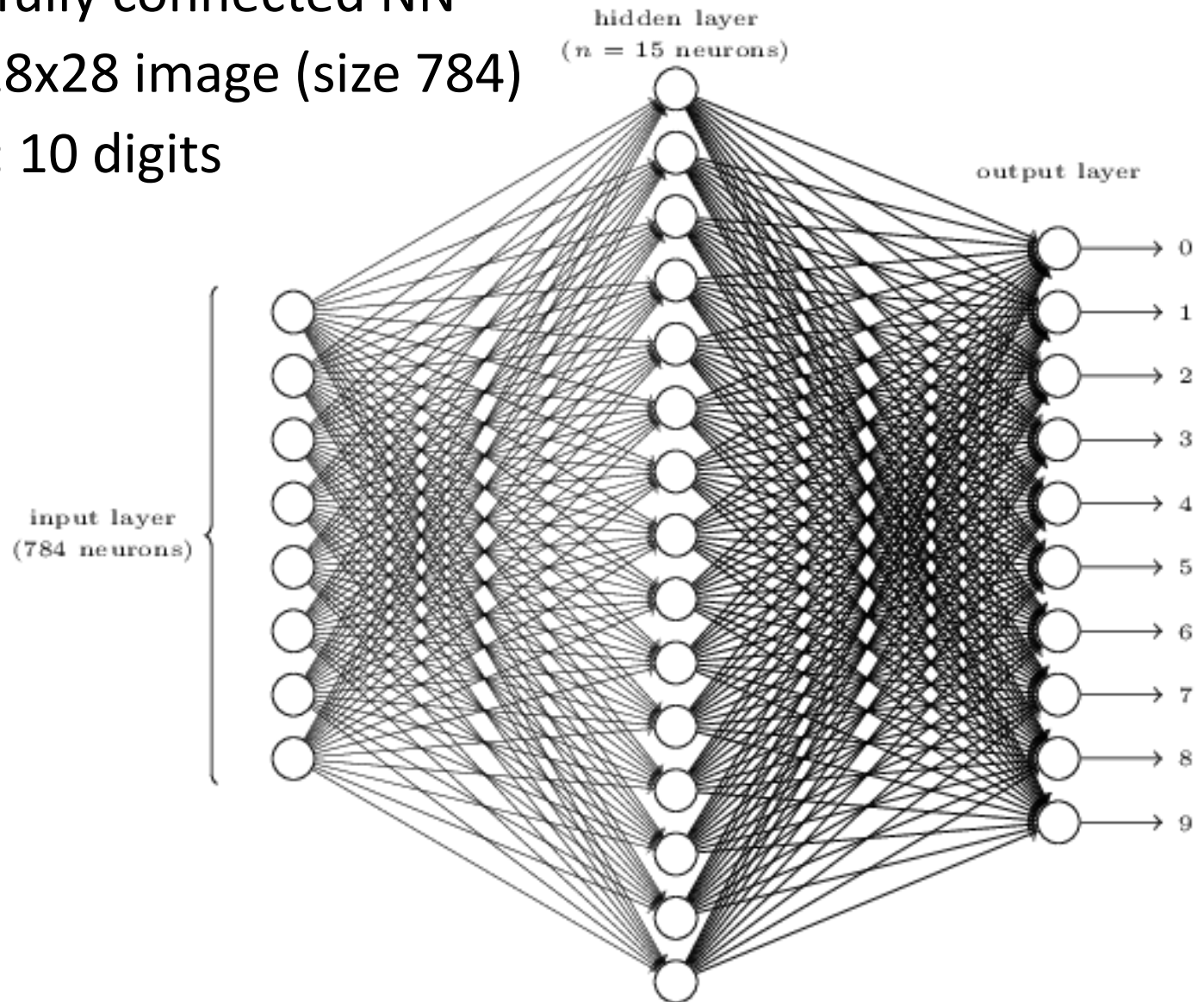


A Convolutional Neural Network (CNN) can achieve an accuracy of 99.67% on MNIST.

Neural network for classify digits

A simple fully connected NN

- Input: 28x28 image (size 784)
- Output: 10 digits



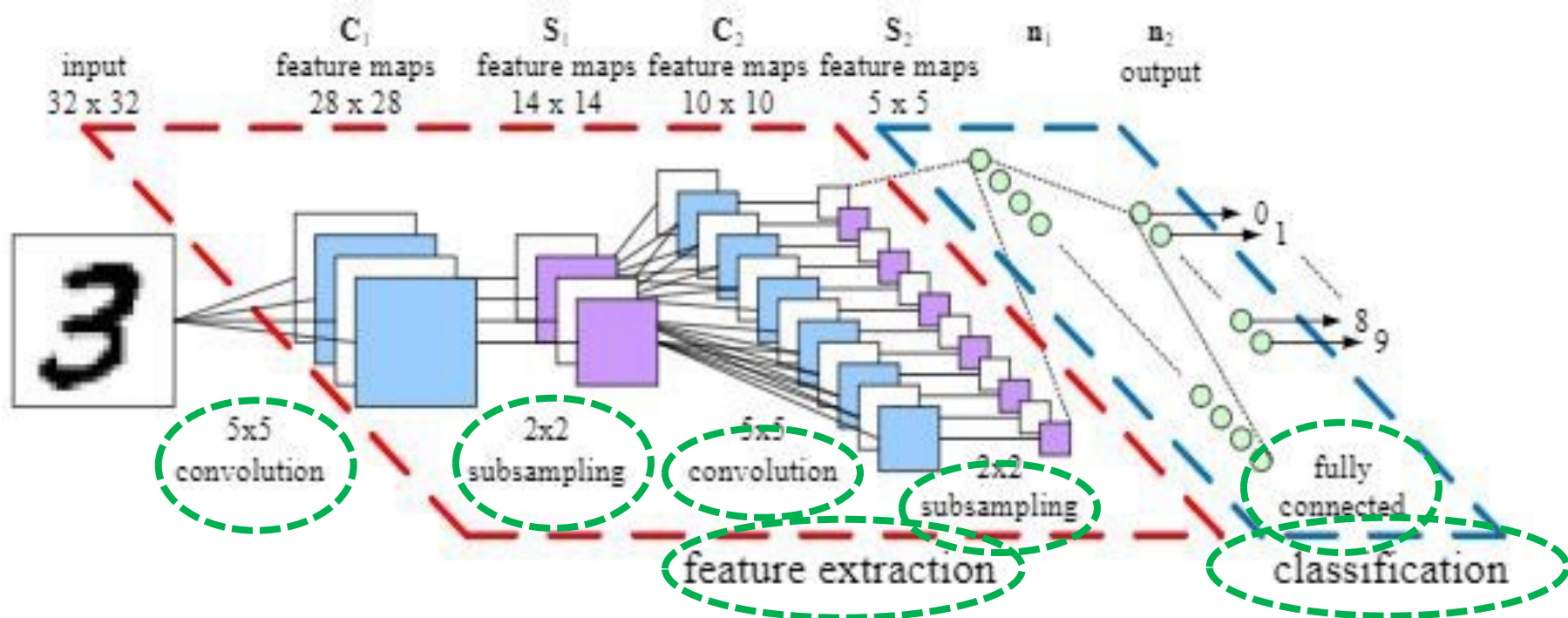
Convolutional Neural Networks (CNNs)

1. What is Computer Vision?
2. Milestones in Computer Vision
3. What is Convolutional Neural Networks (CNNs)?
 - a. **Convolutional layer – Filter**
 - b. Pooling
 - c. Dense layer
4. AlexNet 2012 and ZFNet 2013
5. VNet 2014
6. GoogLeNet 2014
7. ResNet 2015
8. ReduNet 2022
9. References
 1. MIT online course 6.S191 “Introduction to Deep Learning”
 2. Stanford CS230 “Deep Learning”
 3. Chan, Yu, You, Qi, Wright, Ma, ReduNet: A White-box Deep Network from the Principle of Maximizing Rate Reduction, Journal of Machine Learning Research 23 (2022) 1-103

3. Convolutional Neural Network (CNN)

CNN (LeNet 1998) consists of 2 parts (3 kinds of layers):

- Feature extraction (input = image; output = set of features)
Convolutional layer and Pooling (subsampling) layer
- Classification - **Fully connected layer**

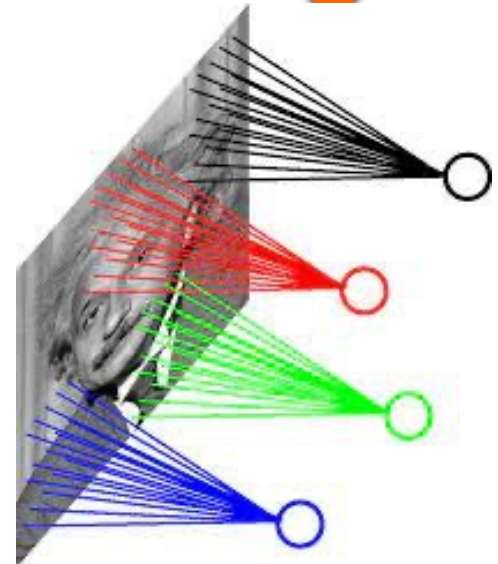
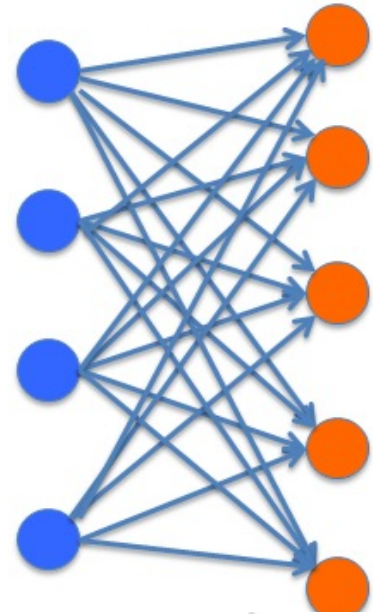


Convolutional Neural Network (CNN)

Fully-connected layer

- lots of **connections**!
Ex: 100x100 image has 10k pixels
100M connections between 2 layers
- each pixel contributes to all other pixels
- For **image**
 - only **adjacent pixels** are relevant
 - same functions for each patch (detection)
- Convolution –
reduces the number of connections to fit the structure of the input?
 - Ex: for 5x5 patch,
number of parameters = 25
number of connections = 25x100x100
(400 times reduction, i.e., 0.25%)

extricates the most important attributes



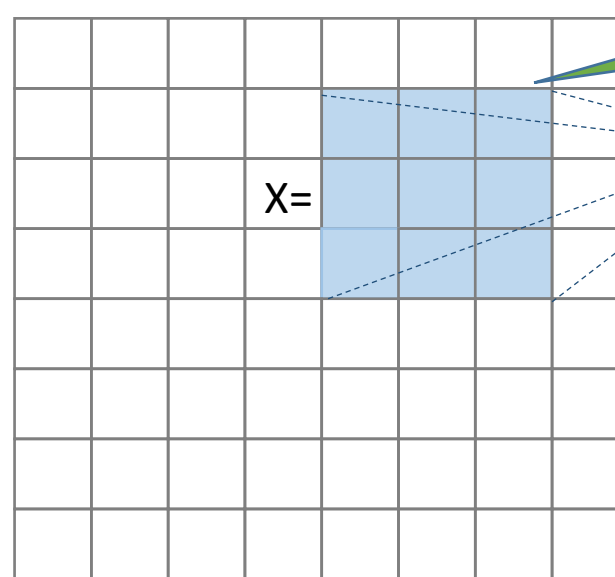
3a. Convolutional Layer - Filters

- **CONV** layer computes the output (hidden neuron) of neurons connected to local contiguous regions
- **(local receptive field (LRF))**, where x is the input values in LRF,
 - portion of input image that a neuron “looks at” or connected to
- Similar to looking at a very small part of the input through the peep hole (**filter or kernel**).
- The input is fully **scanned** from left to right and top to bottom
- **Parameter sharing:** same matrix w and bias b for all LRF

$W =$

w_{11}	w_{12}	w_{13}
w_{21}	w_{22}	w_{23}
w_{31}	w_{32}	w_{33}

Filter (Kernel)



Layer ℓ

Output = $w^T x + b$

$$= \sum_{ij} x_{ij} w_{ij} + b$$

Output value becomes an entry in the hidden layer (*feature map*)

Convolutional Layer - Filters

1	1	0	1	0	0	0	1
1	1	0	1	0	0	1	1
0	1	0	0	0	1	1	0
0	0	1	0	1	1	1	0
0	0	1	0	0	1	0	1
1	1	1	1	1	1	1	0
1	1	0	0	0	1	1	0
1	0	0	0	0	1	1	1

Layer ℓ

1	0	1
0	1	0
1	0	1

Filter
(Kernel)

Convolutional Layer - Filters

1_{x1}	1_{x0}	0_{x1}	1	0	0	0	1
1_{x0}	1_{x1}	0_{x0}	1	0	0	1	1
0_{x1}	1_{x0}	0_{x1}	0	0	1	1	0
0	0	1	0	1	1	1	0
0	0	1	0	0	1	0	1
1	1	1	1	1	1	1	0
1	1	0	0	0	1	1	0
1	0	0	0	0	1	1	1

Layer ℓ

2

Assuming bias $b = 0$

1	0	1
0	1	0
1	0	1

Filter
(Kernel)

Convolutional Layer - Filters

1	1 _{x1}	0 _{x0}	1 _{x1}	0	0	0	1
1	1 _{x0}	0 _{x1}	1 _{x0}	0	0	1	1
0	1 _{x1}	0 _{x0}	0 _{x1}	0	1	1	0
0	0	1	0	1	1	1	0
0	0	1	0	0	1	0	1
1	1	1	1	1	1	1	0
1	1	0	0	0	1	1	0
1	0	0	0	0	1	1	1

Layer ℓ

2	3
---	---

1	0	1
0	1	0
1	0	1

Filter
(Kernel)

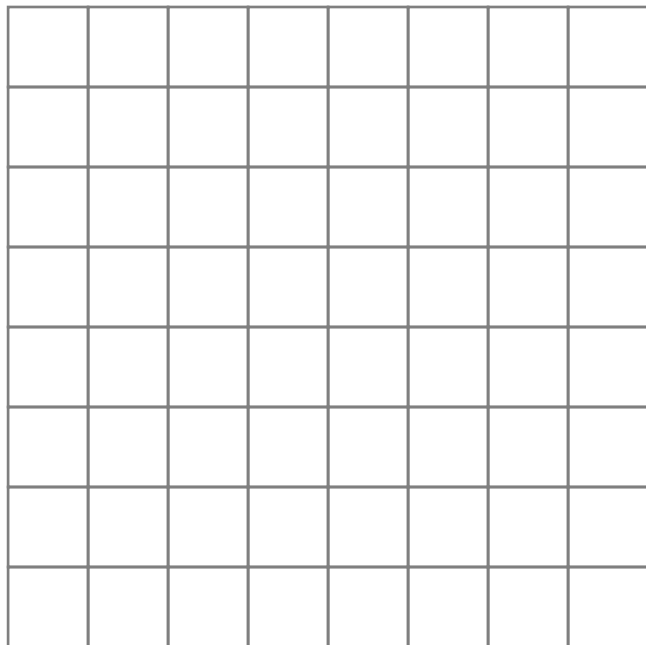
Convolutional Layer - Filters

		w_{11}	w_{12}	w_{13}			
		w_{21}	w_{22}	w_{23}			
		w_{31}	w_{32}	w_{33}			

2	3	
---	---	--

Layer ℓ

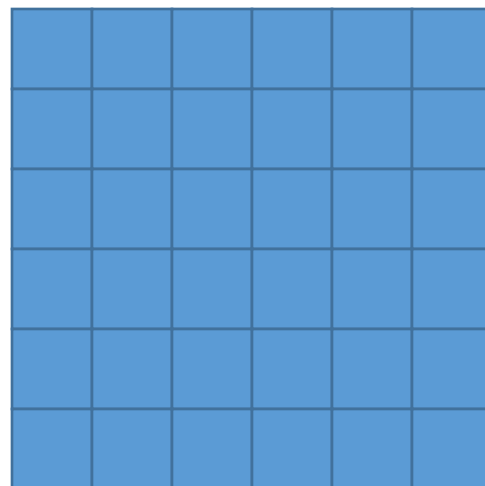
Convolutional Layer - Filters



Layer ℓ
8 x 8 image

w_{11}	w_{12}	w_{13}
w_{21}	w_{22}	w_{23}
w_{31}	w_{32}	w_{33}

3 x 3 filter



Layer $\ell+1$
6 x 6 Feature Map

Convolutional Layer – change **Stride**

w_{11}	w_{12}	w_{13}					
w_{21}	w_{22}	w_{23}					
w_{31}	w_{32}	w_{33}					



Layer ℓ

Convolutional Layer: $S=2$

		w_{11}	w_{12}	w_{13}			
		w_{21}	w_{22}	w_{23}			
		w_{31}	w_{32}	w_{33}			



Layer ℓ

Padding

Zero-padding:

adding P rows on top and bottom,
 P columns on the left and right of the matrix,
filled with zeros.

0	0	0	0	0	0	0	0	0	0
0									0
0									0
0									0
0									0
0									0
0									0
0									0
0									0
0	0	0	0	0	0	0	0	0	0

Zero-Padding
with $P = 1$

Types of Padding

Why?

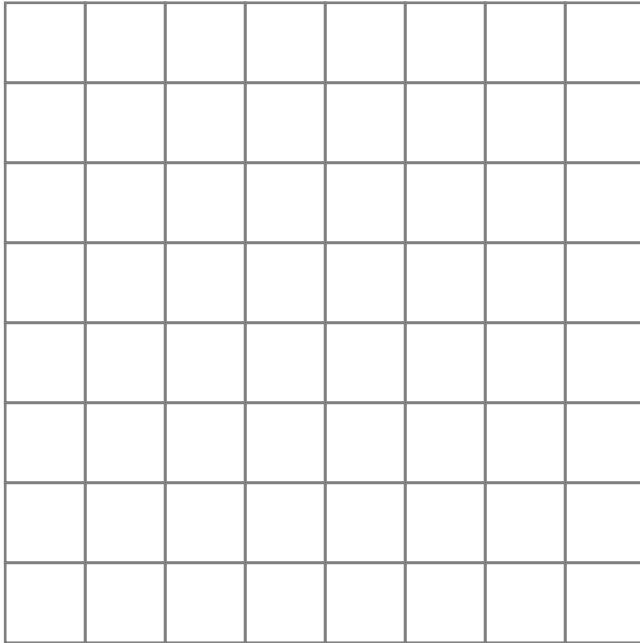
Without padding, the size of the image would be decreased whenever we add a layer.

Thus, limiting the number of total layers.

Types of padding:

- **valid convolution**: no padding, with issues above.
- **same convolution**:
choose P such that image size does not decrease.
- **full convolution**: P is determined so that every pixel is included in k LRFs
(border pixels are usually less represented).
- Optimal padding usually lies between valid and same.

Convolutional Output Layer:



Layer ℓ

Parameters:

- Filter size $F \times F$
- Stride S

$N \times N$ input with padding P

$M \times M$ output

$$M = \lfloor (N + 2P - F) / S \rfloor + 1$$

Output Size?

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

- input 7 x 7 (N x N)
- pad with P = 1 pixel border.
- 3 x 3 (F x F) filter,
- stride (S) = 1
- Output size?

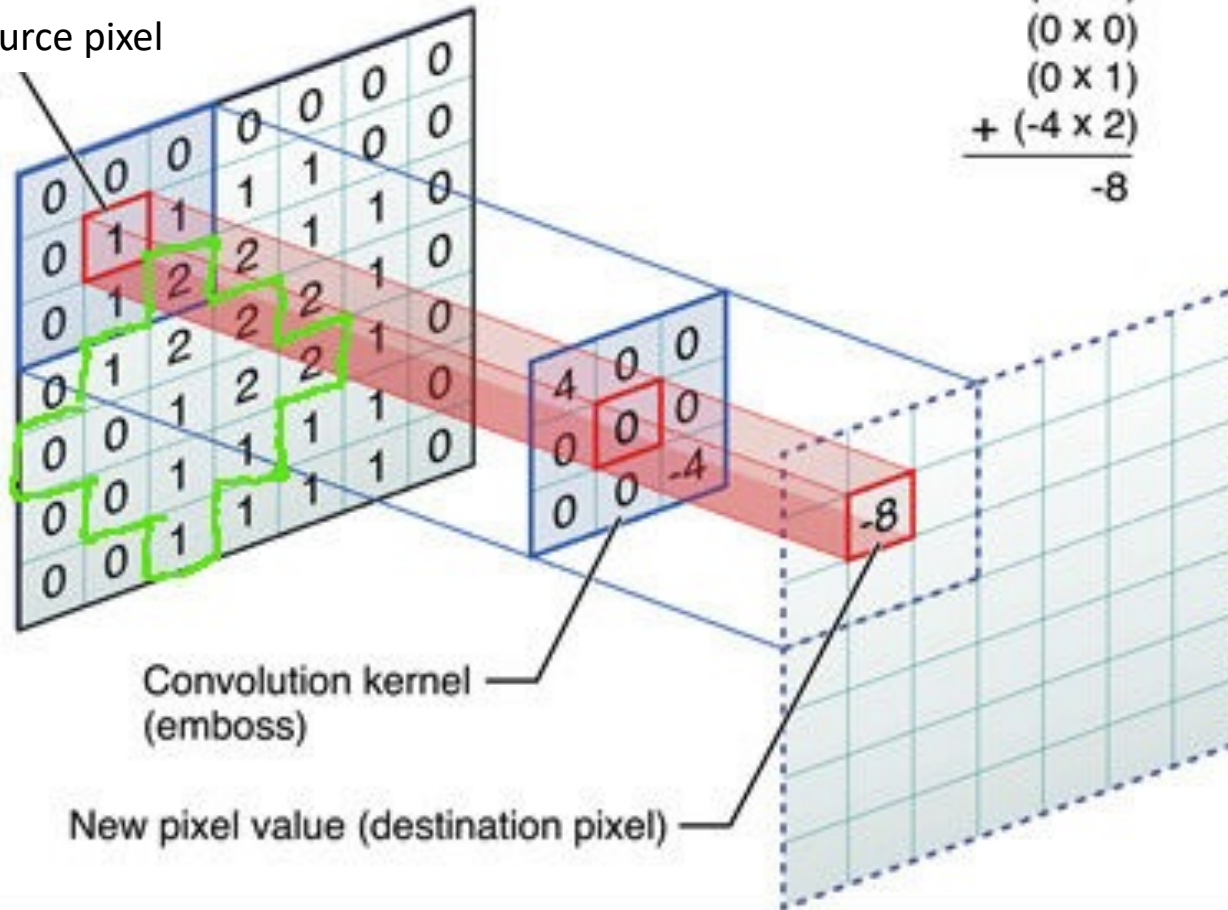
$$M = \lfloor (N + 2P - F) / S \rfloor + 1$$

i.e., 7 x 7

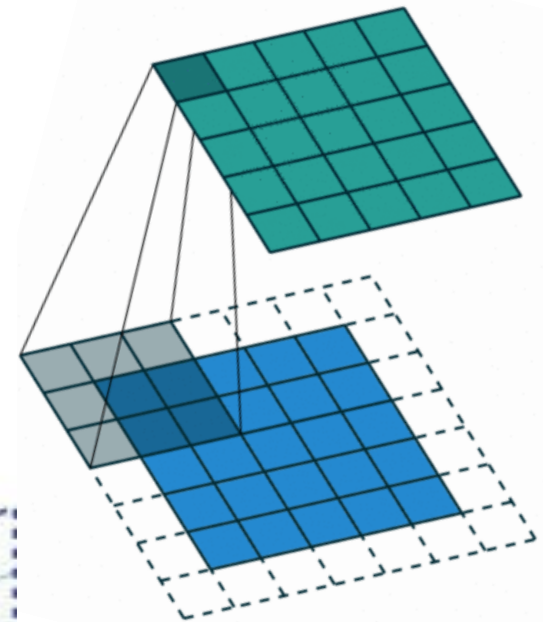
(same convolution)

Center element of the kernel is placed over the source pixel. The source pixel is then replaced with a weighted sum of itself and nearby pixels.

Source pixel



$$\begin{array}{r}
 (4 \times 0) \\
 (0 \times 0) \\
 (0 \times 0) \\
 (0 \times 0) \\
 (0 \times 1) \\
 (0 \times 1) \\
 (0 \times 0) \\
 (0 \times 1) \\
 + (-4 \times 2) \\
 \hline
 -8
 \end{array}$$



Output size

$$M = \lfloor (N + 2P - F) / S \rfloor + 1$$

$$N = 5$$

$$P = 1$$

$$F = 3$$

$$S = 1$$

$$\begin{aligned}
 M &= \lfloor (N + 2P - F) / S \rfloor + 1 \\
 &= \lfloor (5 + 2 \times 1 - 3) / 1 \rfloor + 1 \\
 &= 5
 \end{aligned}$$

Some examples of 3x3 filters

Identity	$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
Edge detection	$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
	$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
	$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	

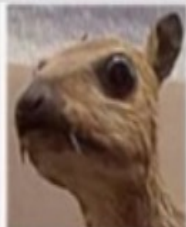
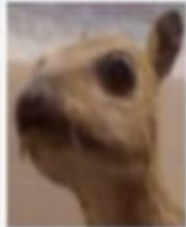
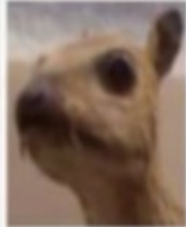
111111111
 111111111
 111111111
 000000000
 000000000
 000000000

000000
 333333
 333333
 000000

Red means -ve

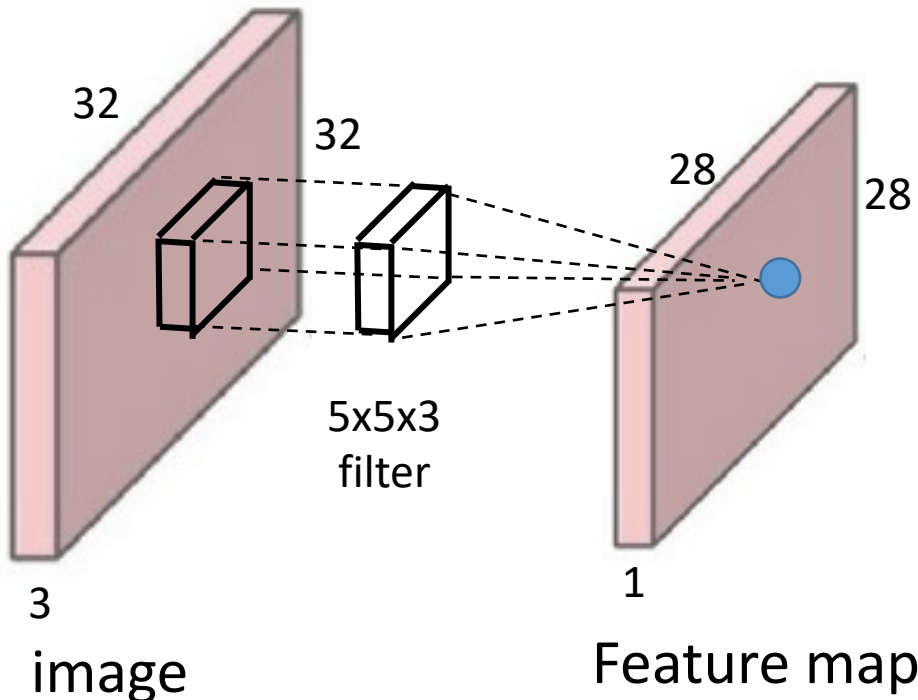


different filters can detect different features

Sharpen	111111 222222 111111 000000	$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
Box blur (normalized)	111111 .666666 .333333 000000	$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
Gaussian blur (approximation)		$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

Convolutional Layer

- Suppose 32×32 color image (red, green, blue)
- Model as a $32 \times 32 \times 3$ cuboid (for detecting feature)



Apply a smaller-size-cuboid **filter** (e.g., $5 \times 5 \times 3$) across input cuboid.

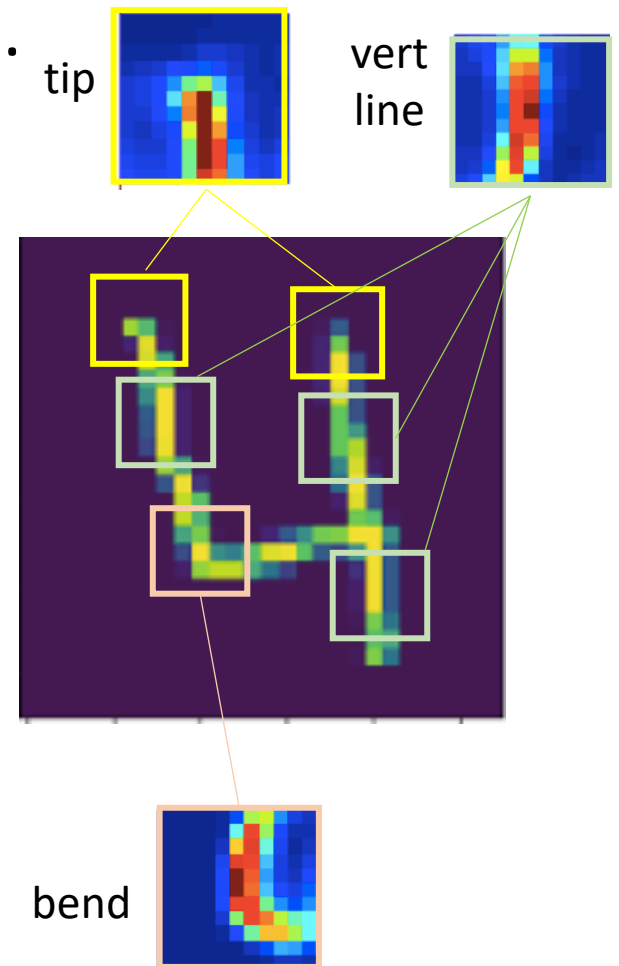
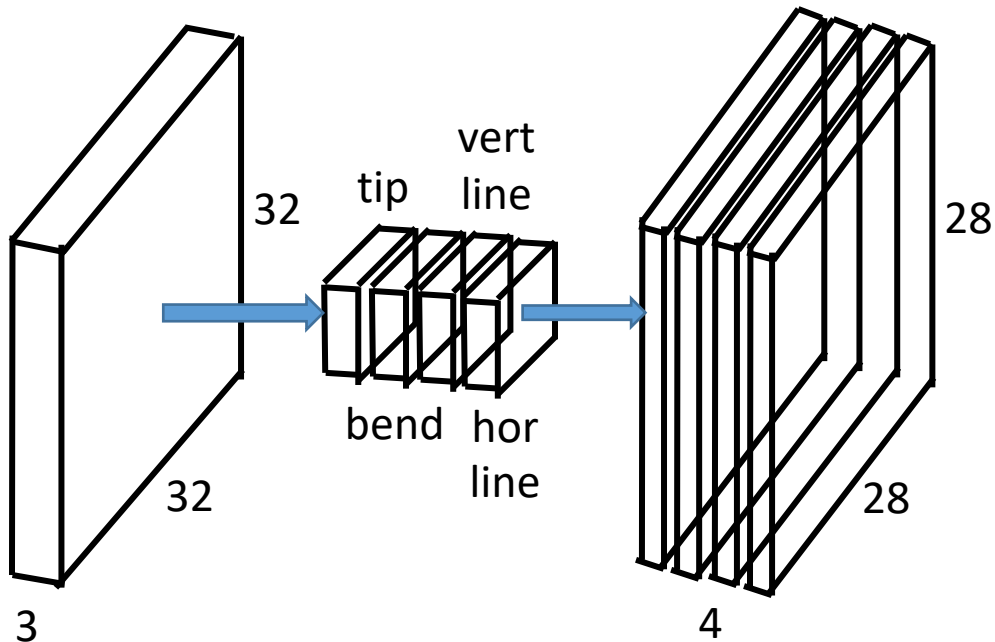
Filter: a set of weights
 $5 \times 5 \times 3 = 75$

$$N = 32, P = 0, F = 5, S = 1$$

$$\begin{aligned} M &= \lfloor (N + 2P - F) / S \rfloor + 1 \\ &= \lfloor (32 + 2 \cdot 0 - 5) / 1 \rfloor + 1 \\ &= 28 \end{aligned}$$

Convolution Layer

With **four** $5 \times 5 \times 3$ filters,
we get **four** $28 \times 28 \times 1$ feature maps (channels),
each looks for different feature
i.e., $28 \times 28 \times 4$ feature maps (**depth** = 4).
Convolution layers learn local patterns
(e.g., vert/hor line, tip, bend)



Input Volume (+pad 1) (7x7x3)

$x[:, :, 0]$

0	0	0	0	0	0	0
0	1	2	0	0	0	0
0	2	2	0	0	0	0

0	2	2	2	1	0	0
0	1	0	0	2	0	0
0	1	1	0	1	0	0
0	0	0	0	0	0	0

$x[:, :, 1]$

0	0	0	0	0	0	0
0	1	0	0	1	0	0
0	1	0	0	0	1	0

0	0	0	1	2	0	0
0	0	1	0	1	0	0
0	1	1	1	1	2	0
0	0	0	0	0	0	0

$x[:, :, 2]$

0	0	0	0	0	0	0
0	0	0	2	2	2	0
0	1	2	2	2	0	0

0	1	1	1	1	0	0
0	1	2	0	0	0	0
0	2	2	2	1	0	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

$w0[:, :, 0]$

0	0	-1
-1	0	0
-1	-1	-1

-4

$w0[:, :, 1]$

1	1	-1
0	0	0
-1	1	1

1

$w0[:, :, 2]$

-1	-1	-1
0	1	1
0	-1	0

-1

Bias b0 (1x1x1)

$b0[:, :, 0]$

1

Output Volume (3x3x2)

$o[:, :, 0]$

-3	-1	4
-2	-7	-4
1	-1	1

$$N = 5, P = 1, F = 3, S = 2$$

$$M = \lfloor (N + 2P - F) / S \rfloor + 1$$

$$= \lfloor (5 + 2 * 1 - 3) / 2 \rfloor + 1$$

$$= 3$$

Input Volume (+pad 1) (7x7x3)

 $x[:, :, 0]$

0	0	0	0	0	0	0
0	1	2	0	0	0	0
0	2	2	0	0	0	0

0	2	2	2	1	0	0
0	1	0	0	2	0	0
0	1	1	0	1	0	0
0	0	0	0	0	0	0

 $x[:, :, 1]$

0	0	0	0	0	0	0
0	1	0	0	1	0	0
0	1	0	0	0	1	0

0	0	0	1	2	0	0
0	0	1	0	1	0	0
0	1	1	1	1	2	0
0	0	0	0	0	0	0

 $x[:, :, 2]$

0	0	0	0	0	0	0
0	0	0	2	2	2	0
0	1	2	2	2	0	0

0	1	1	1	1	0	0
0	1	2	0	0	0	0
0	2	2	2	1	0	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

 $w0[:, :, 0]$

0	0	-1
-1	0	0
-1	-1	-1

 $w0[:, :, 1]$

1	1	-1
0	0	0
-1	1	1

 $w0[:, :, 2]$

-1	-1	-1
0	1	1
0	-1	0

Bias b0 (1x1x1)

 $b0[:, :, 0]$

1

Output Volume (3x3x2)

 $o[:, :, 0]$

-3	-1	4
-2	-7	-4
1	-1	1

Input Volume (+pad 1) (7x7x3)

 $x[:, :, 0]$

0	0	0	0	0	0	0
0	1	2	0	0	0	0
0	2	2	0	0	0	0
0	2	2	2	1	0	0
0	1	0	0	2	0	0
0	1	1	0	1	0	0
0	0	0	0	0	0	0

 $x[:, :, 1]$

0	0	0	0	0	0	0
0	1	0	0	1	0	0
0	1	0	0	0	1	0
0	0	0	1	2	0	0
0	0	1	0	1	0	0
0	1	1	1	1	2	0
0	0	0	0	0	0	0

 $x[:, :, 2]$

0	0	0	0	0	0	0
0	0	0	2	2	2	0
0	1	2	2	2	0	0
0	1	1	1	1	0	0
0	1	2	0	0	0	0
0	2	2	2	1	0	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

 $w0[:, :, 0]$

0	0	-1
-1	0	0
-1	-1	-1

 $w0[:, :, 1]$

1	1	-1
0	0	0
-1	1	1

 $w0[:, :, 2]$

-1	-1	-1
0	1	1
0	-1	0

Bias b0 (1x1x1)

 $b0[:, :, 0]$

1

Filter W1 (3x3x3)

 $w1[:, :, 0]$

-1	0	0
1	-1	-1
0	0	-1

 $w1[:, :, 1]$

-1	0	1
1	-1	1
-1	0	1

 $w1[:, :, 2]$

-1	-1	-1
-1	1	-1
0	1	-1

Bias b1 (1x1x1)

 $b1[:, :, 0]$

0

Output Volume (3x3x2)

 $o[:, :, 0]$

-3	-1	4
-2	-7	-4
1	-1	1

 $o[:, :, 1]$

-7	3	1
-7	-11	-1
-4	-2	-4

Input Volume (+pad 1) (7x7x3)

 $x[:, :, 0]$

0	0	0	0	0	0	0
0	1	2	0	0	0	0
0	2	2	0	0	0	0
0	2	2	2	1	0	0
0	1	0	0	2	0	0
0	1	1	0	1	0	0
0	0	0	0	0	0	0

 $x[:, :, 1]$

0	0	0	0	0	0	0
0	1	0	0	1	0	0
0	1	0	0	0	1	0
0	0	0	1	2	0	0
0	0	1	0	1	0	0
0	1	1	1	1	2	0
0	0	0	0	0	0	0

 $x[:, :, 2]$

0	0	0	0	0	0	0
0	0	0	2	2	2	0
0	1	2	2	2	0	0
0	1	1	1	1	0	0
0	1	2	0	0	0	0
0	2	2	2	1	0	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

 $w0[:, :, 0]$

0	0	-1
-1	0	0
-1	-1	-1

 $w0[:, :, 1]$

1	1	-1
0	0	0
-1	1	1

 $w0[:, :, 2]$

-1	-1	-1
0	1	1
0	-1	0

Bias b0 (1x1x1)

 $b0[:, :, 0]$

1

Filter W1 (3x3x3)

 $w1[:, :, 0]$

-1	0	0
1	-1	-1
0	0	-1

 $w1[:, :, 1]$

-1	0	1
1	-1	1
-1	0	1

 $w1[:, :, 2]$

-1	-1	-1
-1	1	-1
0	1	-1

Bias b1 (1x1x1)

 $b1[:, :, 0]$

0

Output Volume (3x3x2)

 $o[:, :, 0]$

-3	-1	4
-2	-7	-4
1	-1	1

 $o[:, :, 1]$

-7	3	1
-7	-11	-1
-4	-2	-4

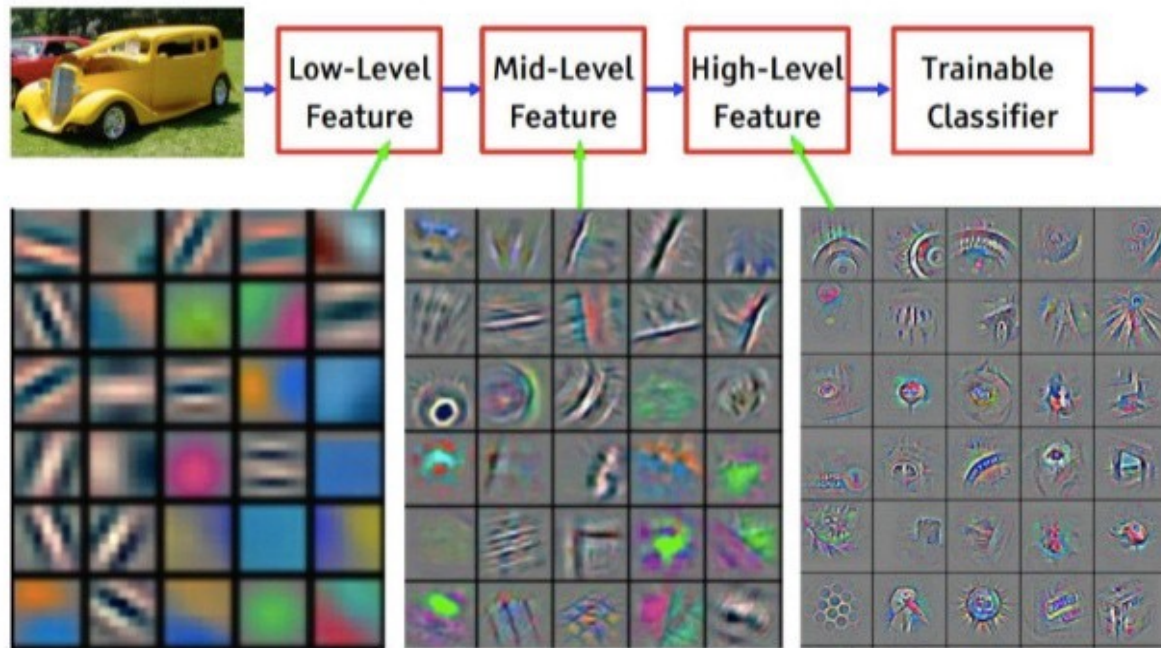
Convolution Layers vs Dense Layers

Translation invariance

- Same filter parameters (local group of values), can recognize the same pattern anywhere.
- Easy for training: fewer training samples

Learn hierarchies of patterns

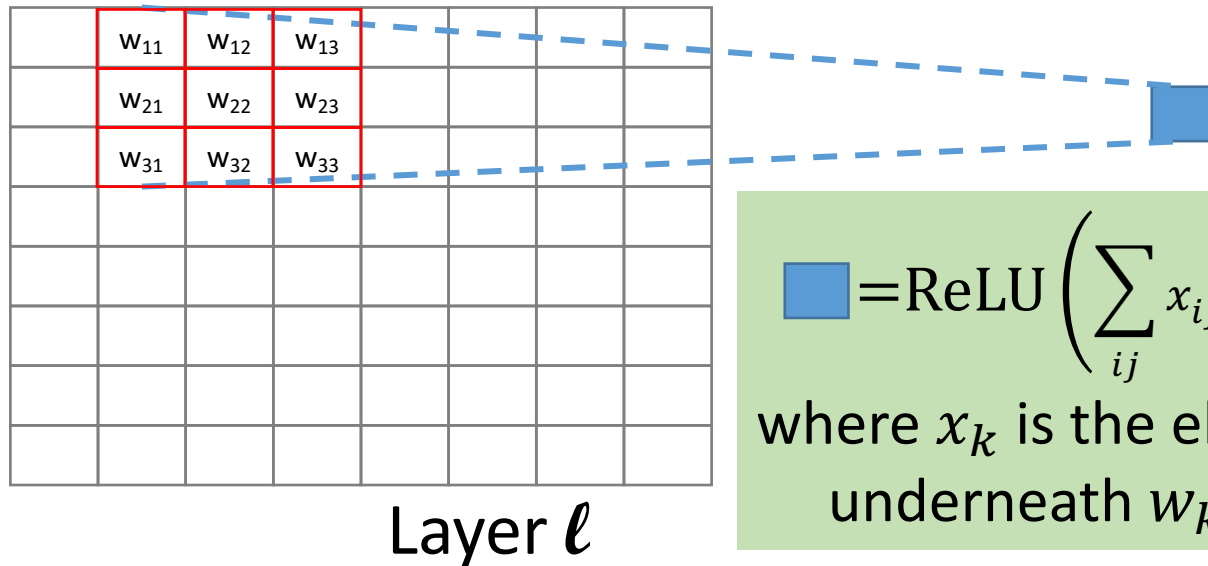
- Low-level layer learns local patterns, e.g., lines, edge,...
- Mid-level layers (larger LRF) learn complex patterns e.g., circles + line
- High-level layers learn more “complex features e.g., wheels, hood,



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

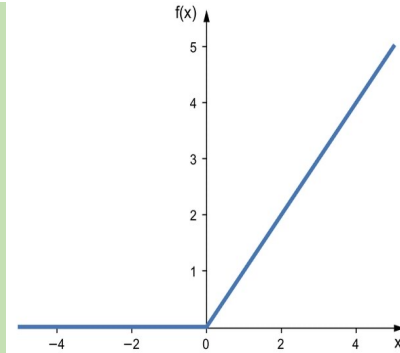
Less parameters

Convolutional Layer - Filters



$$\blacksquare = \text{ReLU} \left(\sum_{ij} x_{ij} w_{ij} + b \right)$$

where x_k is the element underneath w_k

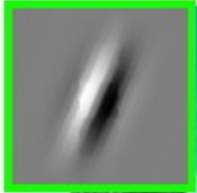


Why ReLu on CNN?

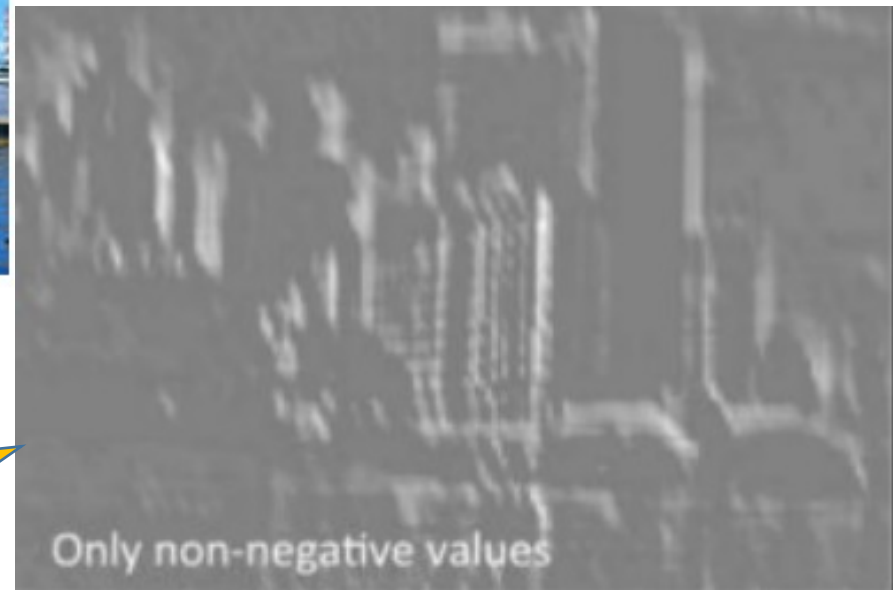
- Computational efficient
- Faster training (no vanishing gradient problem)
- Convolution smooths out borders and changes of colors (black, grey and white) in images
- After rectify the image with ReLu, changes of color are more abrupt.

Edge Detection

After
convolution



After ReLU

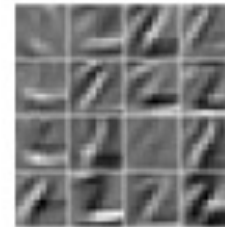




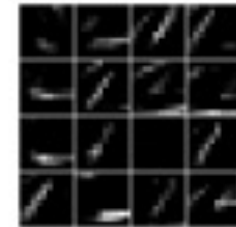
conv1



relu1



conv2



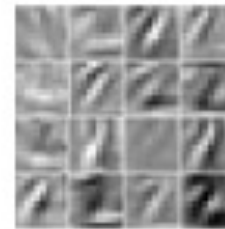
relu2



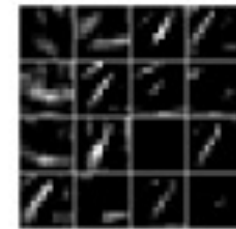
conv1



relu1



conv2



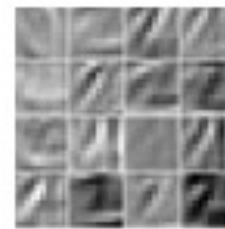
relu2



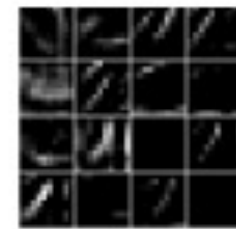
conv1



relu1



conv2



relu2

The structured background has an impact
on the 6 channel responses at the first layer.
Their impact on the 16 channel responses
at the second layer diminishes.

Ref: Understanding convolutional neural networks with a mathematical model, C.-C. Jay Kuo 2016

Convolutional Neural Networks (CNNs)

1. What is Computer Vision?
2. Milestones in Computer Vision
3. What is Convolutional Neural Networks (CNNs)?
 - a. Convolutional layer – Filter
 - b. Pooling**
 - c. Dense layer
4. Image Classification
5. AlexNet 2012 and ZFNet 2013
6. VNet 2014
7. GoogLeNet 2014
8. ResNet 2015
9. ReduNet 2022

References

- MIT online course 6.S191 “Introduction to Deep Learning”
- Stanford CS230 “Deep Learning”
- Chan, Yu, You, Qi, Wright, Ma, ReduNet: A White-box Deep Network from the Principle of Maximizing Rate Reduction, Journal of Machine Learning Research 23 (2022) 1-103

3b. Pooling Layer - reduce size / simplify input

- 2×2 filter, stride = 2
($S < F \rightarrow$ overlapped pooling)
- Maximum or average taken
- Max pooling
- location invariance

Let A is $n \times m$ LRF

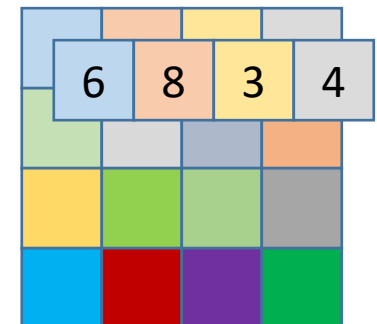
- max pooling: outputs $\max_{ij} A_{ij}$
- L2 pooling: outputs $\|A\|_2$

$$= \sqrt{\sum_{i,j} A_{ij}^2}$$

- average pooling: outputs $\sum_{i,j} A_{ij} / nm$
- Same formula: $M = \lfloor (N + 2P - F) / S \rfloor + 1$
 $= (8 - 2) / 2 + 1 = 4$



Layer l



Layer $l+1$

Why pooling?

- Reduce the number of parameters at the cost of losing the exact position of the feature (merge semantically similar features).
- A 2×2 pooling and stride = 2 maintains a good approximation of the location while decreasing the number of parameters by a factor of 4.

- Having less parameters implies:

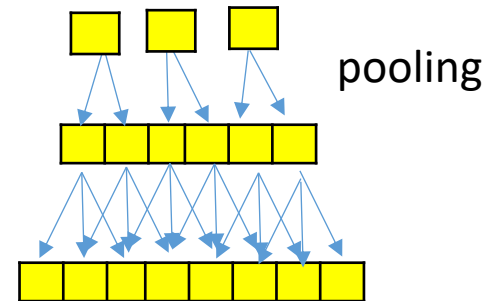
- less memory
- more efficient
- less overfitting

$F = 3 \times 3$, $S = 1$

LRF = 3×3

Pooling 2, $s = 2$

LRF = 4×4



- Allow next convolution layers

Will LRF double for 2×2 pooling?

 - to look at larger windows (local receptive field (LRF)) on input
 - to detect of more complex features higher in the hierarchy.
- Translation invariance: if we move a cat, we still detect it.

Convolutional Neural Networks (CNNs)

1. What is Computer Vision?
2. Milestones in Computer Vision
3. What is Convolutional Neural Networks (CNNs)?
 - a. Convolutional layer – Filter
 - b. Pooling
 - c. Dense layer**
4. Image Classification
5. AlexNet 2012 and ZFNet 2013
6. VNet 2014
7. GoogLeNet 2014
8. ResNet 2015
9. ReduNet 2022
10. References
 1. MIT online course 6.S191 “Introduction to Deep Learning”
 2. Stanford CS230 “Deep Learning”
 3. Chan, Yu, You, Qi, Wright, Ma, ReduNet: A White-box Deep Network from the Principle of Maximizing Rate Reduction, Journal of Machine Learning Research 23 (2022) 1-103

3c. Dense Layers

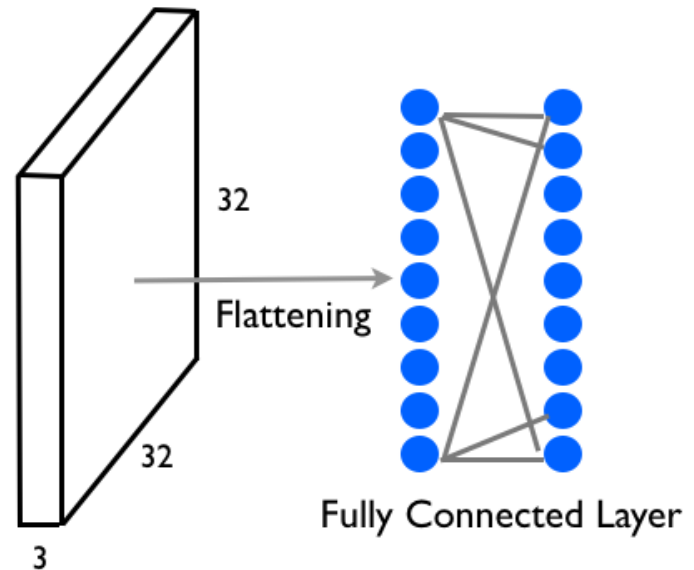
Dense layers, fully connected, are at the end of CNN.

Why dense layers?

- have preserved spatial structures in previous layers, now want to aggregate them together.

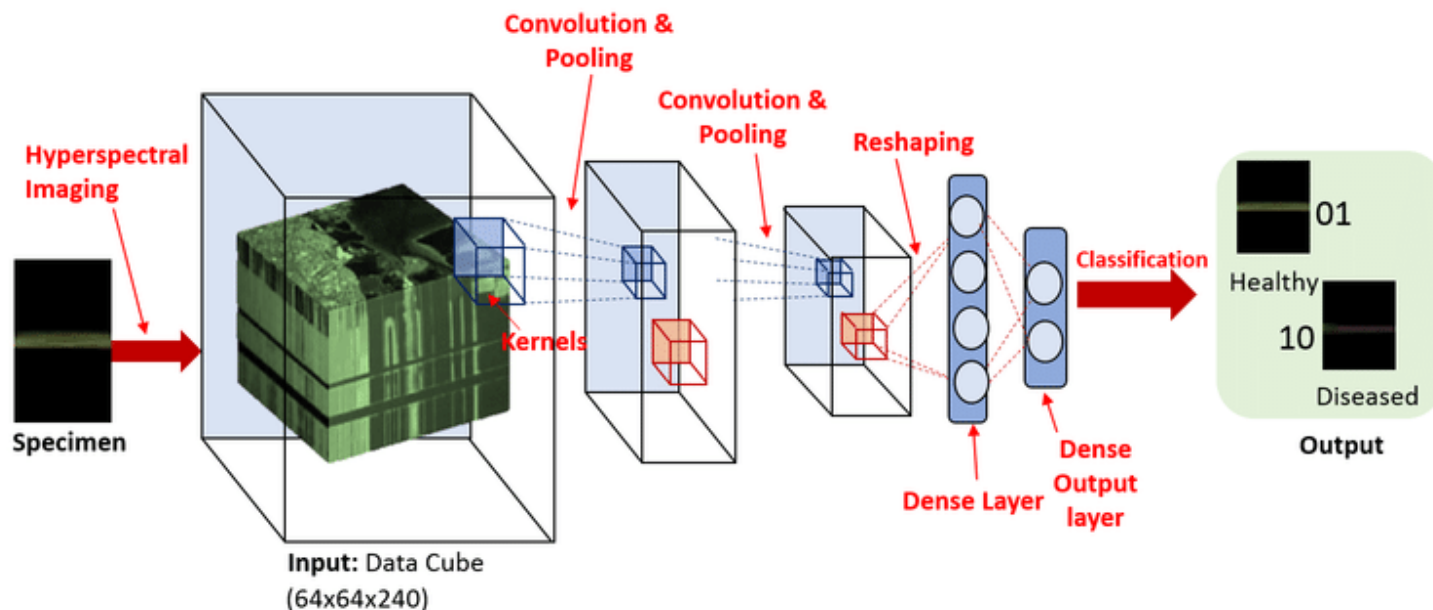
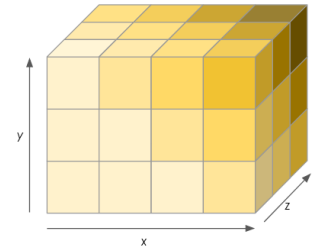
Flattening: convert the last tensor with size $W \times H \times D$ into $W \cdot H \cdot D$ neurons (a one-dimensional vector).

Flattening loses location of the features, however, if we have the head, tail, legs of a cat, then most likely we have a cat.



3D Convolutional Neural Network (3D CNN)

- 2D CNNs operate on 2D data, e.g., images, for all channels
- 3D CNNs use 3D kernel (a cube) over 3D volumetric data (voxel) to detect 3D patterns, e.g., CT/MRI scans capture spatial and/or temporal dependencies in video sequences.



- 3D max-pooling layers and strides to down-sample the spatial dimensions of the data

Data Augmentation for AlexNet

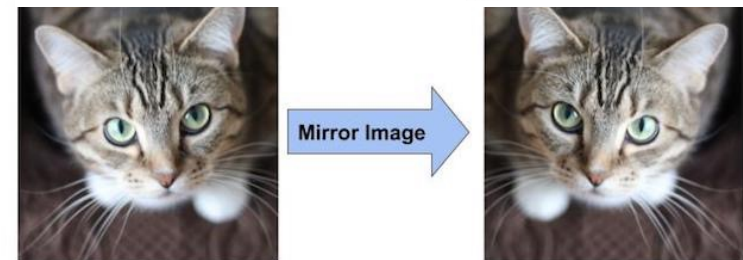
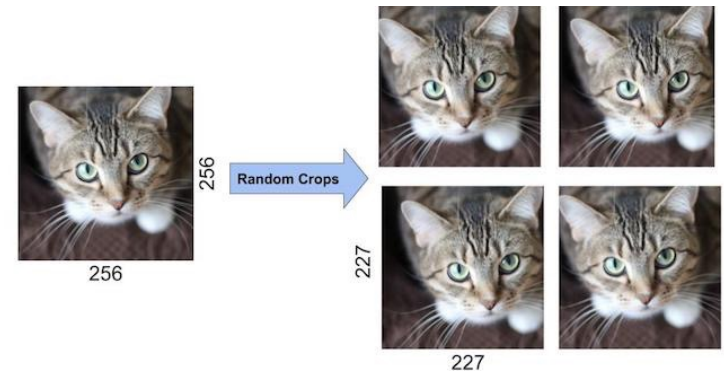
Model might overfit if data is insufficient

Input: RGB image of size 256×256

- resize smaller dimension to 256
- crop resulting image to 256×256
- If needed, replicate a gray scale image to obtain an RGB image

Data augmentation

- randomly crop 227×227 from inside the 256×256 image
- mirroring
- Color augmentation



Original

Positional
Augmentation

Color
Augmentation

Base

